



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:

Компилятор базисного подмножества Рефала-5
с представлением данных,
использующим раскрытые очереди

Студент ИУ9-82Б

(Группа)

(Подпись, дата)

М.А. Аринин

(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

АННОТАЦИЯ

Работа посвящена практической реализации компилятора базисного подмножества языка Рефал-5 с особым вниманием к представлению объектных выражений в памяти. Семейство языков, основанных на переписывании термов, традиционно сталкивается с двумя конкурирующими проблемами: дорогим копированием значений переменных при списковом представлении и дорогой конкатенацией при массивном. Каждая известная реализация решает один из этих компромиссов за счёт другого, и попытка обойти оба сразу оказывается нетривиальной инженерной задачей.

В качестве представления объектных выражений в работе выбрана раскрытая двусторонняя очередь, описанная Окасаки. Эта структура амортизированно даёт константную сложность как для операций на концах выражения, так и для конкатенации, а структурное разделение между версиями устраняет физическое копирование при многократном использовании переменной. Компилятор переводит исходный текст через лексический, синтаксический и семантический анализ во внутреннее представление программы, а исполняющий модуль вычисляет нормализованную форму, опираясь на выбранную структуру данных. Реализация выполнена на языке Java 25 LTS, сборка автоматизирована средствами Apache Maven, корректность подтверждена набором автоматических тестов JUnit 5 и сравнительным прогоном с двумя альтернативными реализациями Рефала-5: классической Refal-5 PZ на языке C и Refal-5J на JVM.

Результаты работы могут использоваться как самостоятельный компилятор базисного подмножества Рефала-5 и как платформа для дальнейших исследований эффективных представлений объектных выражений.

Работа состоит из 64 страниц, включает 4 раздела, 4 приложения, 9 таблиц, 4 рисунка и 36 листингов. Библиографический список содержит 13 источников.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 Исследовательская часть	7
1.1 Программа, функции и предложения	7
1.2 Объектные выражения, термы и символы	7
1.3 Неоднозначное сопоставление	8
1.4 Объектное выражение как абстрактный тип	9
1.5 Классическое (плоское двусвязное) представление	10
1.6 Компромиссное представление с подвешенными скобками	11
1.7 Массивное представление	12
1.8 Векторно-списковое представление	13
1.9 Гипотетическое представление: дерево конкатенаций	14
1.10 Предлагаемое представление: раскрученная двусторонняя очередь	14
1.10.1 Структура мелкого и глубокого уровней	15
1.10.2 Краткое обоснование амортизированных оценок	16
1.10.3 Сводное сравнение представлений	16
1.10.4 Область применимости и ограничения	17
1.11 Выводы по разделу	17
2 Конструкторская часть	19
2.1 Функциональные требования	19
2.2 Архитектура и потоки данных	19
2.3 Внутренние представления данных	20
2.4 Интерфейс контейнера выражений	21
2.5 Декомпозиция на модули	21
2.6 Модель выполнения и точки расширения	22
2.7 Нефункциональные требования	22
2.8 Обработка ошибок и форматы сообщений	23
2.9 Выбор и сокращение встроенной библиотеки	23
2.10 Представление термов и вложенность	23
2.11 Окружение сопоставления и представление подстановок	24
2.12 Требования к механизму сопоставления	24
2.13 Сценарии использования системы	25
3 Технологическая часть	26

3.1 Инструкция пользователя.....	26
3.2 Структура данных: раскрытая двусторонняя очередь	28
3.3 Цикл нормализации	31
3.3.1 Поле зрения и иерархия ViewTerm.....	31
3.3.2 Двустековая реализация нормализатора	32
3.3.3 Оптимизация BulkPassive	34
3.3.4 Укладка правой части правила	35
4 Аналитическая часть.....	38
4.1 Стратегия тестирования.....	38
4.2 Состав тестов	38
4.2.1 Юнит-тесты раскрытой очереди	39
4.2.2 Юнит-тесты лексера, парсера и семантики	39
4.2.3 Юнит-тесты исполнения	40
4.2.4 Интеграционные тесты и тесты совместимости	40
4.3 Покрытие кода	41
4.4 Сравнительные прогоны.....	42
4.4.1 Условия измерений	42
4.4.2 Тестовая программа и методика измерений	42
4.4.3 Результаты Loop/Select	43
4.4.4 Тест на реальной программе: сахахариватель.....	44
4.4.5 Методика и результаты сахахаривателя	45
4.4.6 Обсуждение результатов	45
ЗАКЛЮЧЕНИЕ	48
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	51
ПРИЛОЖЕНИЕ А	53
ПРИЛОЖЕНИЕ Б.....	55
ПРИЛОЖЕНИЕ В.....	62
ПРИЛОЖЕНИЕ Г	64

ВВЕДЕНИЕ

Язык Рефал-5 относится к семейству языков, основанных на переписывании термов: программа задаётся набором правил сопоставления образца с объектным выражением и построения результирующего выражения. На практике стоимость вычислений в таком подходе часто определяется не только числом применённых правил, но и эффективностью представления объектных выражений в памяти, особенно для операций добавления и удаления термов с концов и операции конкатенации.

Выпускная квалификационная работа посвящена разработке компилятора базисного подмножества Рефала-5 с представлением объектных выражений на основе раскрученной двусторонней очереди. Исходный текст проходит лексический, синтаксический и семантический анализ; исполнение реализовано интерпретирующим циклом нормализации без кодогенерации, поэтому в технологической и аналитической частях исполняющая часть системы для краткости именуется интерпретатором. Реализация включает полную цепочку трансляции, автоматическое тестирование и сравнение с двумя альтернативными реализациями языка — классической Refal-5 PZ [1] на языке C и Refal-5J на платформе JVM.

Объектом исследования является программная система, выполняющая трансляцию и исполнение программ на базисном подмножестве Рефала-5. Пользователи системы — разработчики и исследователи, работающие с языком переписывания термов: преподаватели и студенты при изучении формализма, авторы преобразователей программ при отладке выходного кода рассаживателей и других внешних инструментов. Целью работы является разработка компилятора базисного подмножества Рефала-5, использующего раскрученную двустороннюю очередь для представления объектных выражений, и подтверждение его работоспособности автоматическими тестами и сравнительным замером производительности.

Исследовательская часть вводит базовые конструкции и терминологию Рефала-5, а затем разбирает распространённые представления объектных выражений: классическое плоское двусвязное, компромиссное с подвешенными скобками, массивное, векторно-списковое и дерево конкатенаций. У каждого взвешиваются стоимости основных операций и связанные с ними компромиссы; на этом фоне и обосновывается выбор раскрученной двусторонней очереди — структуры, которая амортизированно покрывает все ключевые операции.

Конструкторская часть фиксирует функциональные и нефункциональные требования к системе, описывает её архитектуру, состав модулей и интерфейсы между ними. В этой же части приводятся сценарии использования и определяются форматы диагностических сообщений, что необходимо для последующей реализации.

Технологическая часть посвящена реализации: используется язык Java 25 LTS, сборка автоматизирована средствами Apache Maven [2], тестирование выполнено в JUnit 5 [3]. В этой части подробно разбираются руководство пользователя, реализация раскрученной двусторонней очереди и цикл нормализации с двустековой обработкой поля зрения.

В аналитической части представлены результаты функционального тестирования и сравнительного замера производительности. Помимо автоматических тестов с покрытием кода, выполнены два сравнительных прогона трёх реализаций Рефала-5 — разработанной в работе, классической Refal-5 PZ и Refal-5J. Синтетический тест Loop/Select нагружает конкатенацию растущего аккумулятора и проверяет, совпадает ли измеренное поведение с ожидаемой линейной сложностью раскрученной очереди. Второй прогон — на реальной программе сахараживателя проекта `refal-5-framework` [4] — подтверждает корректность интерпретатора на содержательной нагрузке. Полученные численные результаты сопоставлены с асимптотическими ожиданиями и обсуждены применительно к выбору структуры данных.

Практическим результатом работы является программная система — компилятор базисного подмножества Рефала-5 (Maven-проект `refal5q-impl`), способный выполнять как программы, написанные напрямую на базисном подмножестве, так и результат работы внешнего сахараживателя `r5fw-desugar` проекта `refal-5-framework` [4], обеспечивая запуск программ расширенного синтаксиса Рефала-5 после соответствующего шага предварительной обработки.

1 Исследовательская часть

Раздел вводит базовые понятия Рефала-5 и разбирает, как объектные выражения хранят существующие реализации языка. Сперва — минимум терминологии, на которую опираются дальнейшие рассуждения: программа, функция, предложение, объектное выражение, сопоставление с образцом. Затем — сами представления и их компромиссы, а в итоге обоснование того, почему для проектируемой системы выбрана раскрытая двусторонняя очередь.

1.1 Программа, функции и предложения

Программа на Рефале — это набор функций. Каждая функция задаётся одним или несколькими предложениями (правилами) вида «образец = результат», заключёнными в фигурные скобки. При вызове функции система просматривает её предложения сверху вниз и берёт первое, чей образец сопоставим с аргументом [5, 6]. По подстановке, полученной при сопоставлении, строится результирующее выражение, которое заменяет вызов в текущем поле зрения.

Простейший пример — рекурсивная функция, вычисляющая длину объектного выражения (листинг 1):

Листинг 1 — Функция подсчёта длины выражения

```
1 Length {  
2     = 0;  
3     s.First e.Rest = <Add 1 <Length e.Rest>>;  
4 }
```

Функция `Length` имеет два предложения. Первое срабатывает на пустом выражении и возвращает 0. Второе сопоставляется с непустым выражением: оно связывает первый терм с переменной `s.First`, остаток — с переменной `e.Rest` и рекурсивно подсчитывает длину остатка. Имена с префиксами `s.`, `t.`, `e.` обозначают переменные образца разных типов; их семантика рассматривается ниже.

1.2 Объектные выражения, термы и символы

Все данные в Рефале представлены объектными выражениями. Объектное выражение — это конечная последовательность термов (возможно пустая). Каждый терм имеет один из двух видов:

- атомарный терм — собственно символ в терминологии Рефала; в Рефале-5 различают три разновидности символов: символ-слово (идентифика-

тор: Found, True, Nil), макроцифра (целое число произвольной длины) и литер (один знак строкового литерала);

— скобочный терм — объектное выражение, заключённое в круглые скобки.

Важно не смешивать понятия «символ» и «литера». Символ-слово — это атомарный объект (метка), а не последовательность литер: при сопоставлении Found ведёт себя как один атомарный терм. Запись в одинарных кавычках, например 'abc', обозначает три литеры подряд, то есть объектное выражение длины три. Аналогично, число 42 — это один символ (макроцифра), а не последовательность из двух цифр-литер.

Сопоставление с образцом опирается на три класса переменных образца:

- *s*-переменная (*s.X*) связывается с одним символом — одним атомарным термом любой из трёх разновидностей;
- *t*-переменная (*t.X*) связывается с одним термом, в том числе со скобочным;
- *e*-переменная (*e.X*) связывается с произвольной, возможно пустой, цепочкой термов.

Простой пример с переменной *e*-типа — функция, возвращающая последний символ непустой последовательности атомарных термов (листинг 2):

Листинг 2 — Функция получения последнего символа

```
1 Last {  
2   e.Init s.Last = s.Last;  
3 }
```

При вызове <Last 1 2 3> переменная *e.Init* связывается с выражением 1 2, а *s.Last* — с символом 3. Вариант, в котором *e.Init* забирала бы всё выражение целиком, не годится: тогда для *s.Last* не осталось бы ни одного терма, и сопоставление было бы неудачным. Алгоритм сопоставления учитывает требования к концам образца и подбирает разбиение, при котором все переменные связываются согласованно.

1.3 Неоднозначное сопоставление

Если в образце встречается несколько *e*-переменных, сопоставление становится неоднозначным: одно и то же объектное выражение можно разбить разными способами. Стандартный пример — функция, проверяющая вхождение заданного символа в выражение (листинг 3):

Листинг 3 — Неоднозначное сопоставление при поиске символа

```
1 Contains {  
2   s.Needle e.Left s.Needle e.Right = True;  
3   s.Needle e.Any                      = False;  
4 }
```

Первое предложение требует, чтобы и в начале, и где-то дальше в выражении встретился один и тот же символ `s.Needle`. Образец содержит две *e*-переменные `e.Left` и `e.Right`, и при сопоставлении система перебирает варианты: она начинает с пустого `e.Left`, и если очередной `s.Needle` не оказывается на текущей позиции, удлиняет `e.Left` на один терм и пробует снова. Каждое такое удлинение — это комбинация операций «взять один терм с начала остатка» и «дописать этот терм в конец накапливаемого префикса», то есть отделение начала и конкатенация.

Так даже логически простое правило разворачивается в множество шагов, и на каждом интерпретатор разбирает и собирает фрагменты выражения. Сколько стоит один такой шаг, решает представление выражения в памяти — к нему и переходим.

1.4 Объектное выражение как абстрактный тип

Чтобы говорить о стоимости разных представлений, удобно зафиксировать набор операций, через которые выражается интерпретация любого предложения:

- проверка на пустоту;
- отделение начала: получить первый терм и остаток выражения;
- отделение конца: получить последний терм и остаток выражения;
- конкатенация двух выражений — основная операция при построении результата правила;
- выражение из одного терма (создание сингулярного выражения).

Эти операции вызываются на каждом шаге сопоставления и каждом шаге сборки результата. Например, при переборе *e*-переменной (1.3) последовательно работают отделение начала и конкатенация; при возврате результата правила вида $= \langle F \ s.X \rangle \langle G \ e.Y \rangle$; итоговое выражение собирается из нескольких фрагментов с помощью конкатенации. Поэтому выбор представления — это, по существу, выбор того, какие из перечисленных операций должны быть дешёвыми, а какие могут стать дорогими.

Дальнейший разбор идёт по нарастанию компромисса. Списковые представления дешёвы на концах и в конкатенации, но платят копированием значений

переменных. Массивные и векторно-списковые меняют беду местами: копирование дешевле, зато дорожает конкатенация. Дерево конкатенаций показано как гипотетический промежуточный вариант. Раскрученная двусторонняя очередь, которой раздел заканчивается, амортизированно снимает обе проблемы сразу.

1.5 Классическое (плоское двусвязное) представление

Этот способ представления использовался в большинстве реализаций Рефала вплоть до 1980-х годов и часто называется классическим [7] или плоским двусвязным [8]. Объектное выражение хранится как двусвязный список звеньев, где каждое звено соответствует либо атомарному символу, либо одной из двух структурных скобок. У звена-скобки есть дополнительное поле, указывающее на парную скобку. Контейнер выражения хранит указатели на первое и последнее звено.

Стоимость основных операций при таком представлении:

- отделение начала или конца — $O(1)$: сдвинуть указатель; если крайнее звено — открывающая скобка, можно прыгнуть через поле парной скобки на закрывающую и получить скобочный терм целиком;
- конкатенация двух выражений — $O(1)$: достаточно перенастроить четыре указателя на стыке;
- копирование выражения — $O(N)$, где N — число звеньев в записи (включая все вложенные скобки): необходимо физически создать новый список звеньев.

Дешёвая конкатенация делает это представление удобным для построения результата, но за неё приходится платить копированием: если переменная встречается в правой части предложения чаще, чем в левой, каждое «лишнее» вхождение требует полной копии её значения. Стандартная иллюстрация — функция подстановки из препринта Абрамова и Романенко [8]: для каждого символа выражения по таблице ищется его замена.

Листинг 4 — Подстановка по таблице (пример проблемы копирования)

```
1 Subst {  
2   (e.Tab)           = ;  
3   (e.Tab) s.X e.Rest = <Lookup (e.Tab) s.X> <Subst (e.Tab)  
   e.Rest>;  
4 }
```

Переменная `e.Tab` в левой части появляется один раз, а в правой — дважды (передаётся в `Lookup` и в рекурсивный вызов `Subst`). При классическом представлении это означает, что на каждом шаге рекурсии таблица копируется целиком. Если длина исходного выражения и длина таблицы оба порядка n , общая стоимость становится $O(n^2)$, и попытка программировать в функциональном стиле упирается в эту квадратичную асимптоту. Распространённый «инженерный» обход — так называемый «сквозной просмотр выражения» с двумя стеками, переводящий `Subst` в императивный по духу вариант без многократного использования `e.Tab`; копирований он избегает ценой более тёмного кода и потерянной возможности распараллеливания [8].

1.6 Компромиссное представление с подвешенными скобками

Чтобы смягчить проблему копирования, не отказываясь от списковой основы, было предложено компромиссное представление [7]. Оно используется в реализации Рефала-6 и ряде учебных проектов, а также обсуждается в документации Рефала-05 [6].

В этом представлении звено списка соответствует не отдельному знаку записи, а одному терму верхнего уровня. Скобочный терм хранится особым образом: вместо двух парных звеньев (открывающей и закрывающей скобок) звено содержит ссылку на отдельный кольцевой список, представляющий вложенное выражение. У головы кольца хранится счётчик ссылок.

Стоимость операций по сравнению с плоским списком:

- отделение начала и конца — $O(1)$, как и раньше;
- конкатенация — $O(1)$;
- копирование верхнего уровня — $O(N_{\text{верх}})$, где $N_{\text{верх}}$ — число термов верхнего уровня; вложенные выражения при копировании не дублируются, у их кольцевых списков лишь увеличивается счётчик ссылок.

Применительно к функции `Subst` это даёт ощутимый выигрыш в типичных сценариях: если таблица состоит из пар (`s.Key` `e.Value`), то при копировании `e.Tab` физически дублируются только звенья верхнего уровня, а вложенные пары остаются разделёнными между копиями.

Платой становится более сложное управление памятью: необходимо поддерживать счётчики ссылок при разрушении и сборке выражений и аккуратно обрабатывать ситуации, когда переменная связана со внутренностью скобочно-

го терма, который оказался расшарен (тогда модификация одной «копии» могла бы повредить другие). Кроме того, наличие циклических связей у голов колец усложняет автоматическую сборку мусора.

1.7 Массивное представление

Альтернативный путь, предложенный в работе Абрамова и Романенко [8], состоит в том, чтобы хранить объектное выражение в непрерывном участке памяти (массиве тегированных ячеек). Каждая ячейка содержит либо атомарный символ, либо открывающую или закрывающую структурную скобку. Скобочный терм при таком хранении не требует отдельного указателя: он занимает диапазон ячеек от открывающей до закрывающей скобки.

Главное преимущество массивного представления — дешёвое копирование переменной. Значение e -переменной $e.X$ представляется парой индексов (l, r) внутри массива; копирование переменной — это просто копирование пары чисел, то есть $O(1)$. Это превращает функциональный стиль программирования в Рефале из «дорогого» в «нормальный»: даже многократное использование одной и той же переменной в правой части не приводит к физическому дублированию данных. На функции Subst из 1.5 массивное представление даёт линейную асимптотику без каких-либо ухищрений с императивным переписыванием.

Стоимость основных операций:

- отделение начала и конца — $O(1)$: сдвинуть один из индексов;
- копирование переменной — $O(1)$;
- конкатенация двух выражений длины $|X|$ и $|Y|$ — $O(|X| + |Y|)$: требуется выделить новый массив подходящей длины и физически скопировать в него содержимое обоих фрагментов.

Так возникает проблема конкатенации, симметричная проблеме копирования. Если результат правила собирается в цикле, как в типовой функции отображения

Листинг 5 — Построение результата через рекурсивную конкатенацию

```
1 Map {  
2     = ;  
3     s.X e.Rest = <F s.X> <Map e.Rest>;  
4 }
```

то на n шагах рекурсии возникают n конкатенаций. При наивной массивной реализации каждая из них копирует уже накопленный префикс, и суммарная стоимость становится $O(n^2)$.

В препринте [8] предлагается смягчение — так называемые дырки (запас свободного места) по краям массива. Если к выражению с дыркой справа дописывается короткий фрагмент длины k , то вместо создания нового массива достаточно скопировать только эти k ячеек в свободную область. На сериях коротких приписываний это даёт амортизированно линейное общее время. Однако если дырка исчерпывается, приходится снова выделять новый массив и переносить всё содержимое; гарантию $O(1)$ на произвольных последовательностях операций дырки не дают. Дополнительно в [8] обсуждаются меры по снижению затрат на конкатенацию — распознавание частных случаев, функции с несколькими входами и выходами, использование скобок для структуризации данных.

1.8 Векторно-списковое представление

Векторно-списковое представление, предложенное Скоробогатовым [7], нацелено на то, чтобы сохранить дешёвое копирование переменных от массивного представления и одновременно избежать линейной конкатенации.

Идея состоит в том, чтобы изменить интерфейс между функциями. Аргументы функций передаются как векторы (массивы тегированных ячеек), но результат функции возвращает не как один сплошной вектор, а как список ссылок на фрагменты векторов. Конкатенация фрагментов откладывается: пока вычисление не дошло до точки, где результат должен стать пассивным аргументом другой функции (в частности, попасть внутрь скобочного термина), фрагменты остаются отдельными звеньями.

Применительно к функции `Map` это означает следующее. На каждом шаге рекурсии вместо физического присоединения `<F s.X>` к результату `<Map e.Rest>` формируется лёгкое звено «сначала такой-то фрагмент, потом такой-то». Когда вычисление функции завершено и её результат должен стать пассивным выражением,

ем, накопленный список фрагментов собирается в единый вектор за один проход, что даёт линейное общее время вместо квадратичного.

Платой становится более сложный интерфейс языка сборки и более ответственное управление временем жизни фрагментов: пока на фрагмент есть ссылки в незавершённых вычислениях, исходный вектор нельзя освободить или переиспользовать. Подробное описание представления и связанной системы команд абстрактной Рефал-машины приведено в [7].

1.9 Гипотетическое представление: дерево конкатенаций

Среди представлений, не получивших распространения в реализациях Рефала, интерес представляет дерево конкатенаций. В этом представлении объектное выражение хранится в виде двоичного дерева, где листья — небольшие фрагменты массива, а внутренние узлы соответствуют операциям конкатенации поддеревьев.

Преимущество горе в том, что конкатенация двух выражений сводится к созданию одного нового внутреннего узла, что даёт $O(1)$ с периодической перебалансировкой за $O(\log n)$. Копирование переменной также $O(1)$ — это ссылка на корень поддерева. Слабая сторона — операции на концах: чтобы получить первый или последний терм, нужно спуститься по дереву к крайнему листу, что стоит $O(\log n)$ и зависит от баланса. Для Рефала это особенно неудобно, потому что e -переменные требуют как раз многократного отделения термов с краёв.

В литературе по реализациям Рефала горе обсуждается лишь как теоретическая возможность [6].

1.10 Предлагаемое представление: раскрученная двусторонняя очередь

У каждого из разобранных представлений есть своё слабое место: плоский список — проблема копирования; компромиссный — неполное её устранение и сложное управление памятью; массивное — проблема конкатенации; векторно-списковое — сложный интерфейс и привязка к определённой модели вычисления; горе — дорогие операции на концах. В работе вместо них взята раскрученная двусторонняя очередь из монографии Окаси [9]: она амортизированно закрывает обе ключевые проблемы разом.

Идея структуры опирается на два уровня вложенности.

- Мелкий уровень (Shallow) — буфер фиксированной ёмкости B , организованный как циклический массив. Локальные операции на концах при наличии места выполняются как сдвиг индексов, без выделения памяти.
- Глубокий уровень (Deep) — тройка $\langle front, mid, back \rangle$, где крайние компоненты ($front, back$) — мелкие буферы, а средний (mid) — двусторонняя очередь мелких буферов. Структура определена рекурсивно: средняя очередь сама может быть как мелкой, так и глубокой.

Структуру держит один инвариант: внутренние буферы средней очереди заполнены не менее чем наполовину. Без него глубокая очередь могла бы выродиться в длинную цепочку почти пустых буферов; с ним глубина рекурсивного представления растёт логарифмически от длины выражения.

Стоимость основных операций:

- отделение начала или конца — $O(1)$ амортизированно;
- конкатенация двух выражений — $O(1)$ амортизированно (за счёт перетекания крайних буферов в среднюю очередь);
- использование переменной несколько раз — $O(1)$, благодаря структурному разделению: новая версия выражения ссылается на те же неизменяемые буферы и узлы, что и старая.

Структурное разделение в данном случае — инженерная техника, а не свойство языка: в самой модели Рефала-5 нет изменяемых ячеек памяти, и вычисление концептуально строит новые выражения, а задача реализации в том, чтобы за этой моделью не стояло физическое копирование.

1.10.1 Структура мелкого и глубокого уровней

Рисунок 1 показывает идею мелкого буфера: элементы размещены циклически, концы задаются индексами «head» и «tail»; в общем случае head может находиться правее tail, и разрыв «обходится» по модулю ёмкости B .



Рисунок 1 — Идея мелкого буфера: элементы размещены циклически, концы задаются индексами

Глубокий уровень состоит из двух мелких буферов и средней очереди, которая сама хранит мелкие буферы (рисунок 2). Рекурсивность означает, что средняя

очередь при необходимости тоже становится глубокой, и мы получаем структуру с логарифмической по основанию $B/2$ глубиной.

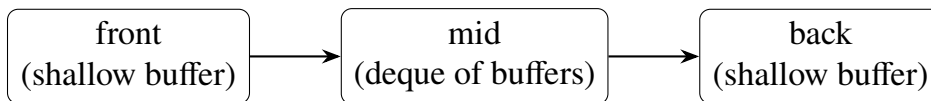


Рисунок 2 — Глубокий уровень раскрученной двусторонней очереди: два мелких буфера и средняя очередь буферов

1.10.2 Краткое обоснование амортизированных оценок

Большинство операций затрагивает только крайние буферы *front* и *back*: пока в них есть свободное место, добавление и удаление сводятся к сдвигу индексов в циклическом массиве и являются строго $O(1)$. Перетекание буфера в среднюю очередь происходит только при достижении границ заполненности и требует $O(B)$. Если каждой такой дорогой операции приписать стоимость, накопленную на предшествующих локальных операциях, то средняя стоимость шага остаётся около константы; это и есть классический амортизированный аргумент [9].

Интуитивно редкие глубокие переносы можно сравнить с цепочкой переносов при прибавлении единицы в позиционной системе счисления: вероятность цепочки длины k убывает геометрически, и вклад в среднюю стоимость образует сходящийся ряд. Для основания $q \geq 2$ ожидаемое число переносов

$$\mathbb{E}[K] = \sum_{k \geq 1} \frac{k(q-1)}{q^{k+1}} = \frac{1}{q-1}.$$

Это иллюстрация, а не строгое доказательство для конкретной реализации. Полные техники амортизированного анализа функциональных структур данных (включая раскрутку и ленивые вычисления) систематически изложены у Окасаки [9].

1.10.3 Сводное сравнение представлений

Сравнение рассмотренных представлений приведено в таблице 1. Зафиксирована стоимость трёх ключевых операций, через которые выражается основная вычислительная нагрузка исполняющего модуля: отделение термина с конца, конкатенация двух выражений и использование одного значения переменной несколько раз.

Таблица 1 — Сравнение представлений объектных выражений по стоимости основных операций

Представление	Концы	Конкатенация	Копирование переменной
Классическое (плоское двусвязное)	$O(1)$	$O(1)$	$O(N)$ (вся запись)
Компромиссное (подвешенные скобки)	$O(1)$	$O(1)$	$O(N_{\text{верх}})$
Массивное (с дырками)	$O(1)$	$O(X + Y)$, аморт. линейная при послед. приписываниях	$O(1)$
Векторно-списковое	$O(1)$	откладывается, $O(1)$ на сборке	$O(1)$
Дерево конкатенаций	$O(\log n)$	$O(1)$, перебалансировка $O(\log n)$	$O(1)$
Раскрученная двусторонняя очередь	$O(1)$ аморт.	$O(1)$ аморт.	$O(1)$

1.10.4 Область применимости и ограничения

Раскрученная двусторонняя очередь оправдана прежде всего тогда, когда выражения достаточно длинные, а в правилах активно используются e -переменные, порождающие многократные разбиения и сборки. На коротких выражениях и в программах, где почти каждое правило связано с одним-двумя термами, выигрыш от сложной структуры может уступить простым спискам или массивам из-за более высокого постоянного множителя.

Целевой реализацией в данной работе является среда JVM. Это естественно сочетается с неизменяемыми объектами и структурным разделением, но накладные расходы на работу сборщика мусора нужно учитывать при выборе ёмкости B мелкого буфера: слишком малое B приводит к большому числу мелких объектов, слишком большое — ухудшает локальность модификаций функционального стиля.

1.11 Выводы по разделу

Требования к представлению объектных выражений базисного подмножества Рефала-5 сводятся к следующему: дешёвые операции на концах, дешёвая конкатенация при построении результата и отсутствие физического копирования при многократном использовании одной переменной. Существующие представления закрывают часть этих требований: классическое и компромиссное списковые

— быстрые на концах и при конкатенации, но дорогие на копировании; массивное и векторно-списковое — быстрые на копировании, но имеют ограничения по конкатенации или по интерфейсу языка сборки; горе — быстрая конкатенация и копирование, но дорогие операции на концах. Раскрученная двусторонняя очередь покрывает все три требования амортизированно за $O(1)$ и поэтому выбрана в качестве представления объектных выражений в проектируемой системе.

2 Конструкторская часть

Конструкторская часть фиксирует архитектуру системы: какие модули в неё входят, как они связаны и какие данные передают друг другу. Отправных точек две — выбранное представление выражений (раскрученная двусторонняя очередь) и грамматика базисного подмножества; от них исходит вся дальнейшая декомпозиция системы.

Начинается часть с требований — функциональных и нефункциональных, — а дальше требования разворачиваются в цепочку этапов обработки, внутренние представления данных, интерфейс контейнера выражений и, наконец, декомпозицию на модули с обработкой ошибок и составом встроенной библиотеки. Связь требований с конкретными модулями держится в поле зрения на всём протяжении раздела.

2.1 Функциональные требования

Функциональные требования следуют непосредственно из постановки задачи: система должна принимать и исполнять программы на базисном подмножестве Рефала-5, сообщая об ошибках входного текста. К системе предъявляются требования:

- приём исходного текста на подмножестве Рефала-5;
- выдача сообщений об ошибках с привязкой к позиции во входном тексте;
- выполнение программы в модели «выбор первого применимого правила» для функций;
- сокращённая библиотека встроенных функций (только необходимый минимум по постановке).

Из этих требований и складывается состав модулей. Приём текста и диагностика ошибок ложатся на лексический и синтаксический анализаторы, хранящие координаты токенов; модель «выбора первого применимого правила» воплощается в семантическом анализе и нормализаторе; сокращённая библиотека встроенных функций задаёт границу того, какие программы система обязана исполнять. Нефункциональные требования (переносимость, воспроизводимость сборки) рассматриваются отдельно в п. 2.5 и далее.

2.2 Архитектура и потоки данных

Система включает цепочку этапов: подготовка входа (при необходимости), лексический анализ, синтаксический анализ, семантический анализ и выполнение

правил. Представление объектных выражений едино для этапов выполнения и используется при сопоставлении и сборке результата.

На практике удобно разрешить внешний «рассахариватель», который переводит исходный текст полного Рефала-5 к синтаксису базисного подмножества (условия, блоки и др.), чтобы не расширять грамматику минимальной системы. Такой шаг не меняет семантику рассматриваемых программ при условии корректности рассахаривания, но выносит сложные конструкции из ядра транслятора. В реализации в качестве рассахаривателя используется внешний инструмент `r5fw-desugar` из проекта `refal-5-framework` [4]; сам интерпретатор его не запускает, но принимает на вход результат его работы (см. п. 2.13).



Рисунок 3 — Этапы обработки программы и основные данные

Целевой средой реализации выбрана платформа JVM, конкретно — Java 25 LTS [10]. Это естественно сочетается с представлением программы и выражений как ссылочных структур и со сборкой мусора, но накладные расходы GC должны учитываться при выборе гранулярности объектов (термы, буферы раскрученной двусторонней очереди, узлы дерева программы).

2.3 Внутренние представления данных

Каждому этапу обработки соответствует своя структура данных, а граница между этапами — это смена представления. В проектировании фиксируются следующие структуры:

- токены с координатами (строка/столбец) для точной диагностики;
- дерево программы как набор функций и правил сопоставления;
- объектные выражения как двусторонняя очередь термов.

Выбор каждой структуры продиктован требованиями. Координаты в токенах нужны, чтобы сообщения об ошибках указывали точное место во входном тексте. Дерево программы хранит функции и их предложения в форме, удобной для сопоставления образца с аргументом без повторного разбора исходного текста. Объектные выражения вынесены в отдельный контейнер по другой причине: от стоимости операций над ними (конкатенация, добавление и удаление на

концах) зависит производительность всего интерпретатора, поэтому им отведён отдельный интерфейс, описанный далее.

2.4 Интерфейс контейнера выражений

Для независимости модулей выделяется интерфейс контейнера выражения, содержащий операции: проверку пустоты, отделение терма с начала/конца, добавление/удаление на концах, итерацию и конкатенацию. Конкретная реализация (мелкий и глубокий уровни раскрученной двусторонней очереди) скрыта за этим интерфейсом и может заменяться без изменения логики сопоставления. В Java-реализации это запечатанный интерфейс `Expr<T>` с двумя реализациями — `ShallowDeque<T>` и `DeepDeque<T>`; такая форма позволяет компилятору проверять полноту разбора случаев в `switch`-выражениях и используется в реализации операции `concat` (см. раздел 3).

2.5 Декомпозиция на модули

С практической точки зрения систему удобно декомпозировать на набор модулей со строго определёнными ответственностями. Для удобства сопоставления проекта и реализации модули названы по соответствующим Java-пакетам:

- `refal.lexer` — лексический анализатор: преобразование потока символов в поток токенов с координатами (`Lexer`, `Token`, `TokenType`, `LexException`);
- `refal.parser` — синтаксический анализатор: построение дерева программы (`Parser`, `ParseException`);
- `refal.ast` — определения узлов дерева программы (`Term` как запечатанный интерфейс, наследники `Sym/Num/Str/Var/Br/Call`, а также `Rule`, `Func` и `Program`);
- `refal.semantic` — семантический анализатор: проверка определённости функций, согласованности типов переменных и т. п. (`SemanticChecker`, `SemanticException`, `SemanticInfo`);
- `refal.deque` — основная реализация контейнера выражения (`Expr`, `ShallowDeque`, `DeepDeque`, вспомогательный класс `ExprOps` с реализацией конкатенации);
- `refal.runtime` — слой исполнения: сопоставление образца с выражением, подстановка результата, цикл нормализации, библиотека встроенных функций (`Runtime`, `Matcher`, `Substitutor`, `Bindings`, `Builtins`, `BuiltinFn`, `RefalException`);

— `refal.Main` — интерфейс командной строки.

Семантический анализ выделен в отдельный этап, так как иначе диагностика ошибок класса «вызов неопределённой функции» либо «связь *s*- и *e*-переменных под одним именем» приходилась бы на стадию интерпретации и затрудняла локализацию причины ошибки.

Связанность при такой декомпозиции остаётся низкой: уточнение синтаксиса входного языка не задевает слой контейнера выражений, а смена представления выражений — логику синтаксического анализа.

2.6 Модель выполнения и точки расширения

Модель выполнения фиксируется как «вызов функции → просмотр предложений сверху вниз → первое успешное сопоставление». Отдельные точки расширения в проектировании:

- добавление новых классов термов (при необходимости расширить подмножество);
- расширение механизма сопоставления (например, введение дополнительных ограничений или условий);
- расширение библиотеки встроенных функций без изменения ядра сопоставления;
- возможное добавление оптимизаций (индексация предложений по первым термам, мемоизация вызовов).

Сами оптимизации в задачи этого этапа не входят, но важно, что выбранный интерфейс контейнера выражений их не блокирует: добавить индексацию предложений или мемоизацию можно будет без переделки ядра.

2.7 Нефункциональные требования

Помимо функциональности, для системы важны следующие свойства:

- предсказуемость потребления памяти: представление выражений должно использовать структурное разделение и избегать полного копирования;
- диагностика: сообщения об ошибках должны указывать координаты во входном тексте и причину ошибки;
- расширяемость: состав встроенных функций допускается сокращать/расширять без изменения ядра сопоставления;

- переносимость: выбор языка реализации со сборкой мусора и поддержкой обобщённых типов; в качестве конкретной платформы выбрана Java 25 LTS [10], что обеспечивает запуск на любой ОС с установленной JVM;
- воспроизводимость сборки: сборка и тестирование автоматизированы средствами Apache Maven [2]; зависимости (JUnit 5 [3], AssertJ [11], Checkstyle [12], JaCoCo [13]) фиксируются версиями в pom.xml.

2.8 Обработка ошибок и форматы сообщений

Ошибки целесообразно разделять на классы: лексические (неожиданный символ, незакрытый строковый литерал), синтаксические (неожиданная конструкция, нарушение структуры определения функции), семантические (неопределённая функция, несовместимые типы переменных, вызов в образце, отсутствие определения для \$ENTRY) и ошибки времени выполнения (отсутствие применимого правила, превышение лимита шагов). Для повышения качества диагностики в проектировании закладывается единый формат сообщения: координата (строка, столбец), тип ошибки и краткое описание. Для каждой категории определён собственный класс исключения (LexException, ParseException, SemanticException, RefalException), сообщение которого начинается с координат вида line:col:.

2.9 Выбор и сокращение встроенной библиотеки

Сокращённая библиотека встроенных функций проектируется как отдельный слой среды выполнения: каждая встроенная функция имеет строго определённый интерфейс, а таблица примитивов отделена от механизма сопоставления. В Java-реализации интерфейс BuiltinFn имеет единственный метод apply(Runtime rt, List<Term> args), возвращающий список термов (раскрытие реализовано интерпретатором). Регистрация выполняется в Builtins.defaultBuiltins(): изменение или сокращение состава встроенных функций сводится к редактированию этой таблицы и не затрагивает ядро сопоставления и нормализации.

2.10 Представление термов и вложенность

В выбранном подмножестве термы образуют минимально необходимый набор для выражения большинства «структурных» преобразований: атомарные термы (символы-слова, числа, строковые литералы) и скобочные термы. Скобочный терм содержит вложенное выражение, что фактически вводит деревоподобную структуру данных поверх линейной последовательности. С точки зрения проекти-

рования это означает, что контейнер выражения должен хранить не только атомы, но и ссылки на вложенные выражения, причём вложенность может быть произвольной.

Здесь важно различать две роли контейнера:

- на верхнем уровне контейнер обслуживает операции над последовательностью термов (концы, конкатенация, разбиение);
- на уровне скобок контейнер используется рекурсивно, так как содержимое скобок является самостоятельным выражением.

Одна и та же структура работает на всех уровнях вложенности одинаково — это упрощает архитектуру и делает поведение системы предсказуемое.

2.11 Окружение сопоставления и представление подстановок

Результатом сопоставления образца и выражения является окружение (подстановка), связывающая переменные образца со значениями. В проектировании выделяются три класса связываний:

- *s*-переменная связывается с атомарным термом (число, символ-слово или строковый литерал);
- *t*-переменная связывается с термом (включая скобочный);
- *e*-переменная связывается с выражением (цепочкой термов).

С практической точки зрения удобно хранить подстановку в виде отображения «имя переменной → значение», где значение для *e*-переменных является контейнером выражения. Это позволяет строить результат без распаковки цепочек в списки/массивы: результат собирается из фрагментов через операции конкатенации и добавления на концах, опираясь на интерфейс контейнера. В Java-реализации эту роль выполняет класс `Bindings` (см. приложение 4.4.6).

2.12 Требования к механизму сопоставления

Чтобы разделить «логику сопоставления» и «операции над данными», проектирование фиксирует требования к модулю сопоставления:

- вход: образец (последовательность термов с переменными) и выражение-аргумент;
- выход: либо «неуспех», либо подстановка, корректная относительно ограничений типов переменных;

- свойство: если сопоставление успешно, то подстановка совместима со всеми повторными вхождениями переменных (одно имя — одно значение).

Такая специфика позволяет в дальнейшем заменять стратегию перебора границ *e*-переменных (например, на более эффективную), не меняя остальную архитектуру. При этом контейнер выражения должен обеспечивать операции, которые нужны сопоставлению: проверку равенства фрагментов, получение префикса/суффикса и сборку новых выражений с разделением структуры.

2.13 Сценарии использования системы

Проектирование исходит из основного сценария — пакетный запуск: чтение программы из файла, запуск входной функции с заданным выражением и вывод результата. В реализации этому соответствует команда

Листинг 6 — Базовая схема запуска

```
1 java -jar target/refal5q.jar <file.ref>[+lib.ref...] [--  
   entry Name] [--step-limit N] [--expr "args..."]
```

Для программ, использующих расширенный синтаксис Рефала-5 (условия, *e*.Cond, блоки { : }), предусмотрен сценарий с предварительным запуском внешнего рессахаривателя *r5fw-desugar* [4]: исходный файл сначала проходит через рессахариватель, после чего полученный файл подаётся на вход интерпретатору. Сам интерпретатор рессахариватель не запускает; такой подход выносит сложные синтаксические преобразования за пределы ядра системы.

Интерактивный режим (последовательные запуски с разными входными выражениями без повторной загрузки программы) в рамках данного проектирования не фиксируется; его можно рассматривать как возможную доработку или расширение — например, для отладки и исследования поведения правил.

3 Технологическая часть

В этой части описаны три аспекта реализации: руководство пользователя (что поставить, как собрать проект под Apache Maven и как запустить интерпретатор), устройство центральной структуры данных — раскрученной двусторонней очереди на запечатанных типах Java — и цикл нормализации, в котором поле зрения обрабатывается на двух стеках. Руководство идёт первым, чтобы запуск системы можно было воспроизвести ещё до разбора внутренностей; язык реализации — Java 25 LTS [10].

3.1 Инструкция пользователя

Чтобы собрать и запустить интерпретатор, на машине должны стоять JDK, Maven и (только для построения графиков) Python 3; конкретные версии перечислены в таблице 2. Установленные версии удобно проверить командами `java --version` и `mvn --version`.

Таблица 2 — Требования к среде разработки

Компонент	Минимальная версия	Примечание
Java Development Kit	25 LTS	Сборка и запуск
Apache Maven	3.9.x	Сборка проекта
Python 3	3.9+	Только для построения графиков

Проект собирается командой Apache Maven из корня репозитория `refal5q-impl/`:

Листинг 7 — Команда сборки

```
1 mvn package -DskipTests
```

Результат сборки — файл `target/refal5q.jar`, самодостаточный JAR-архив с прописанным классом `Main-Class: refal.Main`. Все зависимости рантайма включены в него через плагин `Maven Shade`, поэтому для запуска достаточно стандартной JVM без дополнительных библиотек.

Сборка с полным прогоном тестов выполняется командой `mvn package` (без флага `-DskipTests`) или `mvn verify`, которая дополнительно формирует отчёт о покрытии кода инструментом `JaCoCo`.

Собранный интерпретатор запускается стандартной командой `java -jar`:

Листинг 8 — Форма запуска

```
1 java -jar target/refal5q.jar [OPTIONS] program.ref
```

Позиционный аргумент — путь к исходному файлу .ref. Поддерживаемые опции перечислены в таблице 3.

Таблица 3 — Опции командной строки

Опция	Описание
<code>--entry <имя></code>	Явное указание входной функции; имеет приоритет над <code>\$ENTRY</code> в файле
<code>--step-limit <N></code>	Максимальное число шагов нормализации (по умолчанию — 100 000)
<code>--expr <терм...></code>	Начальное выражение, передаваемое входной функции вместо пустого

Код завершения процесса отражает результат выполнения (таблица 4).

Таблица 4 — Коды завершения процесса

Код	Условие
0	Успешное завершение или <code><Exit 0></code>
1	Ошибка лексического, синтаксического или семантического анализа
2	Ошибка времени выполнения: нет подходящего правила, превышение лимита шагов
<i>n</i>	Значение <i>n</i> , переданное встроенной функции <code><Exit n></code>

Пример запуска программы hello.ref с явным указанием точки входа:

Листинг 9 — Пример запуска

```
1 java -jar target/refal5q.jar --entry Go hello.ref
```

Полный набор тестов запускается одной командой Maven:

Листинг 10 — Запуск тестов

```
1 mvn test
2 mvn verify
3 mvn test -Dtest=AutotestsRunnerTest
```

Первые две команды прогоняют 143 JUnit-теста, покрывающих все компоненты проекта. Третья команда запускает отдельный класс `AutotestsRunnerTest`, который вызывает интерпретатор на 81 внешней программе из двух наборов: автотесты репозитория `refal-5j` и десахаренные тесты репозитория `refal-5-framework`. Каждая программа выполняется в отдельном процессе с тайм-аутом 20 секунд.

Отчёт JaCoCo формируется в каталоге `target/site/jacoco/`; открыть его можно браузером, указав путь к `index.html`.

3.2 Структура данных: раскрученная двусторонняя очередь

Объектное выражение в нашей реализации хранится в раскрученной (bootstrapped) двусторонней очереди по Окасаки [9]. У неё амортизировано константная конкатенация — та самая операция, на которой классические списочные интерпретаторы Рефала (включая Refal-5 PZ и Refal-5J) тратят линейное время.

Контейнер выражения объявлен как обобщённый запечатанный интерфейс `Expr<T>` с двумя реализациями:

Листинг 11 — Запечатанный интерфейс `Expr<T>`

```
1 public sealed interface Expr<T> extends Iterable<T>
2     permits ShallowDeque, DeepDeque {
3     boolean isEmpty();
4     int size();
5     PeekResult<T> separateFront();
6     PeekResult<T> separateBack();
7     Expr<T> pushFront(T x);
8     Expr<T> pushBack(T x);
9     static <T> Expr<T> empty()          { return
ShallowDeque.empty(); }
10    static <T> Expr<T> singleton(T x) { return
ShallowDeque.<T>empty().pushBack(x); }
11    static <T> Expr<T> concat(Expr<T> a, Expr<T> b) {
return ExprOps.concat(a, b); }
12 }
```

Слово `sealed` запрещает любые другие наследники. Компилятор Java 25 на этом основании проверяет полноту разбора в `switch`-выражениях, поэтому в коде конкатенации мы можем перечислить ровно четыре случая по типам операндов и не писать ветку `default`: её отсутствие гарантировано системой типов.

Поведение обеих реализаций задаётся двумя константами. Ёмкость мелкого буфера — `ShallowDeque.CAPACITY = 9`, нижняя граница заполненности `MIN_FILL = 5`, то есть $\lceil CAPACITY/2 \rceil$. Нечётное значение выбрано по арифметическим соображениям: при переполнении мелкого буфера суммарно получается $CAPACITY + 1 = 10$ элементов; они делятся ровно пополам на два фрагмента по 5, и оба фрагмента сразу удовлетворяют `MIN_FILL`. С чётной ёмкостью пришлось бы либо мириться с остатком в один элемент, либо добавлять отдель-

ную обработку этого случая — ни то ни другое не даёт выигрыша по свойствам структуры.

На практике глубина рекурсии `DeepDeque` при таком `CAPACITY` почти не выходит за два уровня: в тестовых программах с выражениями длиной до десятков тысяч термов средняя очередь сама ни разу не успевает стать глубокой.

Мелкий уровень — это `ShallowDeque<T>`, иммутабельный класс с тремя полями: массив фиксированной длины `Object[] buf` размером `CAPACITY`, индекс головы `head` и текущий размер `sz`. Доступ по логическому индексу i выполняется через единственную формулу:

Листинг 12 — Циклическое индексирование в `ShallowDeque`

```
1 int idx(int i) {  
2     return Math.floorMod(head + i, CAPACITY);  
3 }
```

`floorMod` здесь стоит вместо обычного `%`, потому что в Java оператор `%` при отрицательном делимом возвращает отрицательный остаток, а нам нужен индекс в массиве. `pushFrontTight` как раз сдвигает голову на -1 , и без `floorMod` в этой точке пришлось бы вручную добавлять `CAPACITY`.

В классе определены два варианта операций добавления:

- `pushFrontTight(T)` и `pushBackTight(T)` — внутренние, возвращают `ShallowDeque<T>` и бросают `IllegalStateException` при переполнении; применяются там, где вызывающий код гарантирует наличие свободного места (например, при формировании буферов во время конкатенации);
- `pushFront(T)` и `pushBack(T)` — публичные (методы интерфейса `Expr<T>`); при переполнении автоматически переходят в глубокий уровень через `DeepDeque.mkDeep`.

Глубокий уровень — `DeepDeque<T>`; он повторяет в коде ту самую тройку $\langle front, mid, back \rangle$, что описана в п. 1.10:

Листинг 13 — Поля `DeepDeque<T>`

```
1 public final class DeepDeque<T> implements Expr<T> {  
2     private final ShallowDeque<T>          front;  
3     private final Expr<ShallowDeque<T>>    mid;  
4     private final ShallowDeque<T>          back;  
5     private final int                       sz;  
6     ...
```

Вся «раскрутка» спрятана в одном типе: поле `mid` объявлено как `Expr<ShallowDeque<T>`, а не как конкретный класс. Поэтому средняя очередь хранит мелкие буферы, но сама подчиняется тому же интерфейсу и может на следующем уровне снова оказаться `DeepDeque`. Именно эта рекурсия по типу и даёт два (на практике — редко больше) уровня вложенности без какого-либо особого кода для каждого из них.

Конструктор `mkDeep` автоматически схлопывает глубокую очередь обратно в мелкую, когда суммарное число элементов не превышает `CAPACITY`:

Листинг 14 — Конструктор `mkDeep` с откатом в `ShallowDeque`

```

1 static <T> Expr<T> mkDeep(ShallowDeque<T> front,
2                           Expr<ShallowDeque<T>> mid,
3                           ShallowDeque<T> back, int total
4     ) {
5     if (total <= CAPACITY) {
6         ShallowDeque<T> flat = ShallowDeque.empty();
7         for (T x : iterableOf(front)) flat = flat.
8         pushBackTight(x);
9         for (ShallowDeque<T> bucket : iterableOf(mid))
10            for (T x : iterableOf(bucket)) flat = flat.
11            pushBackTight(x);
12         for (T x : iterableOf(back)) flat = flat.
13         pushBackTight(x);
14         return flat;
15     }
16     return new DeepDeque<>(front, mid, back, total);
17 }
```

Параметр `total` передаётся явно, чтобы не пересчитывать его внутри: `mid.size()` возвращает число буферов в средней очереди, а не суммарное число листовых элементов, поэтому наивная формула `front.size() + mid.size() + back.size()` оказалась бы некорректной. Каждый вызов `mkDeep` обновляет `total` инкрементально — прибавляя или вычитая единицу по ходу операции добавления или извлечения.

Главная операция, ради которой выбрана эта структура, — конкатенация. Она реализована в служебном классе ExprOps и разбирает четыре случая по мелкости/глубине операндов:

Листинг 15 — Разбор четырёх случаев конкатенации

```
1 static <T> Expr<T> concat(Expr<T> a, Expr<T> b) {
2     if (a.isEmpty()) return b;
3     if (b.isEmpty()) return a;
4     if (a instanceof ShallowDeque<T> sa && b instanceof
        ShallowDeque<T> sb)
5         return concatSS(sa, sb);
6     if (a instanceof ShallowDeque<T> sa && b instanceof
        DeepDeque<T> db)
7         return concatSD(sa, db);
8     if (a instanceof DeepDeque<T> da && b instanceof
        ShallowDeque<T> sb)
9         return concatDS(da, sb);
10    DeepDeque<T> da = (DeepDeque<T>) a;
11    DeepDeque<T> db = (DeepDeque<T>) b;
12    return concatDD(da, db);
13 }
```

В случае Deep + Deep пограничные буферы a.back и b.front склеиваются в один буфер, а concat вызывается рекурсивно уже на средних очередях. На каждом уровне рекурсии работа ограничена константой, а сама рекурсия идёт не глубже логарифма от длины выражения — так Окасаки и получает амортизированную константу [9].

3.3 Цикл нормализации

3.3.1 Поле зрения и иерархия ViewTerm

Нормализация выражения выполняется итеративным проходом по полю зрения — линейной последовательности элементарных единиц. Каждая такая единица описывается запечатанным интерфейсом ViewTerm:

Листинг 16 — Иерархия ViewTerm

```
1 public sealed interface ViewTerm
2     permits ViewTerm.Passive, ViewTerm.BulkPassive,
3             ViewTerm.AngOpen, ViewTerm.AngClose,
4             ViewTerm.ParenOpen, ViewTerm.ParenClose {
5 }
```

```

6      record Passive(Term term)                implements ViewTerm
      {}
7      record BulkPassive(Expr<Term> items) implements
ViewTerm {}
8      record AngOpen(String fn)                implements ViewTerm
      {}
9      record AngClose()                       implements ViewTerm
      {}
10     record ParenOpen()                      implements ViewTerm
      {}
11     record ParenClose()                    implements ViewTerm
      {}
12 }

```

Термы-атомы (Sym, Num, Str) и скобочные термы Br после нормализации вложенных выражений представляются как Passive. Вызов <f args> разворачивается в последовательность AngOpen(f), элементы аргументов, AngClose. Скобочный терм (e.X) аналогично разворачивается через ParenOpen и ParenClose. BulkPassive отвечает за оптимизацию связывания переменных и описан в следующем подразделе.

3.3.2 Двустековая реализация нормализатора

Нормализация работает с двумя стеками — left и right:

- right содержит ещё не обработанные элементы поля зрения;
- left накапливает уже пассивные элементы — те, в которых не осталось ни одного активного вызова.

Исходное выражение сначала сплющивается в right слева-направо. Затем главный цикл извлекает элементы из right по одному с головы. Пассивные элементы (Passive, BulkPassive) и открывающие маркеры (AngOpen, ParenOpen) просто перекладываются в left. Обработка наступает при встрече закрывающего маркера:

Листинг 17 — Обработка закрывающего маркера вызова функции

```

1 case ViewTerm.AngClose ignored -> {
2     bumpStep();
3     Expr<Term> argsExpr = Expr.empty();
4     String fn = null;
5     while (true) {
6         ViewTerm top = left.pollLast();

```

```

7         if (top instanceof ViewTerm.AngOpen ao) { fn = ao.
fn(); break; }
8         if (top instanceof ViewTerm.Passive pt)
9             argsExpr = argsExpr.pushFront(pt.term());
10        else if (top instanceof ViewTerm.BulkPassive bp)
11            argsExpr = Expr.concat(bp.items(), argsExpr);
        // O(1)
12    }
13 }

```

При встрече `AngClose` мы снимаем элементы с хвоста `left` до парного `AngOpen` и попутно собираем выражение аргументов `argsExpr`. Одиночный `Passive` присоединяется через `pushFront` за $O(1)$, а `BulkPassive` — через `Expr.concat(bp.items(), argsExpr)`, тоже за $O(1)$ амортизированно.

После сборки аргументов для встроенной функции результат сплющивается и помещается обратно в `right`; для пользовательской функции находится первое подходящее правило и его правая часть, с подставленными связываниями переменных, укладывается на `right` методом `pushResultToStack`.

Закрывающая скобка `ParenClose` обрабатывается по той же схеме сборки с хвоста `left`, но проще: имя функции искать не нужно, а собранные термы не уходят в вычисление, а сразу заворачиваются в скобочный терм `Br`:

Листинг 18 — Сборка скобочного терма по `ParenClose`

```

1 case ViewTerm.ParenClose ignored -> {
2     Expr<Term> brItems = Expr.empty();
3     while (true) {
4         ViewTerm top = left.pollLast();
5         if (top == null)
6             throw new RefalException("Unmatched ')' in
view field");
7         if (top instanceof ViewTerm.ParenOpen) break;
8         if (top instanceof ViewTerm.Passive pt)
9             brItems = brItems.pushFront(pt.term());
10        else if (top instanceof ViewTerm.BulkPassive bp)
11            brItems = Expr.concat(bp.items(), brItems);
12    }
13    left.addLast(new ViewTerm.Passive(new Br(brItems)));
14 }

```

Здесь видна симметрия двух закрывающих маркеров. `AngClose` и `ParenClose` оба сматывают `left` до своего парного открывающего маркера и оба используют один и тот же приём накопления (`pushFront` для одиночного терма, `Expr.concat` для `BulkPassive`). Различие только в том, что получается на выходе: `AngClose` отдаёт собранные термы как аргументы вызова и запускает редукцию, а `ParenClose` оборачивает их в один `Passive` с термом `Bv` и возвращает обратно на `left`. Готовый скобочный терм с этого момента пассивен и больше в нормализацию не вовлекается — ровно поэтому вложенные скобки нормализуются изнутри наружу: внутренний `ParenClose` успевает свернуться раньше, чем до своего хвоста доходит внешний.

Отдельно стоит отметить защиту от рассогласованного поля зрения. Если при сборке скобок стек `left` опустеет раньше, чем встретится `ParenOpen`, это значит, что во входе была лишняя закрывающая скобка — и нормализатор бросает `RefalException` с понятным сообщением вместо того, чтобы молча выдать неверный результат. Симметричная проверка стоит и в `AngClose`. На корректно разобранной программе такие ситуации недостижимы (баланс скобок и угловых скобок гарантируется ещё синтаксическим анализом), но в коде нормализатора они оставлены как явные инварианты: при будущих изменениях нарушение баланса проявится сразу и в точке возникновения, а не где-то ниже по вычислению.

Цикл устроен итеративно, а не рекурсивно, и это снимает зависимость от размера стека JVM: сколь угодно глубоко вложенная рефал-программа обрабатывается на постоянной глубине Java-стека, а расход памяти определяется только длиной поля зрения.

3.3.3 Оптимизация `BulkPassive`

Без оптимизации подстановка значения переменной в правой части правила требовала поэлементной укладки: каждый `Term` из связанного значения помещался на стек `right` как отдельный объект `Passive`:

Листинг 19 — Наивная укладка переменной (до оптимизации)

```
1 case refal.ast.Var v -> {
2     Expr<Term> val = bindings.get(v.key());
3     // Цикл O(N): каждый терм оборачивается в Passive
4     for (Term t : val) {
5         right.addFirst(new ViewTerm.Passive(t));
6     }
```

При следующей встрече `AngClose` те же N элементов снимались с `left` по одному. Получается ровно проблема копирования из п. 1.5: на программе, где аккумулятор растёт от шага к шагу, поэлементная укладка превращает линейный по смыслу шаг в квадратичную суммарную работу.

Чинится это одним изменением: вместо перебора элементов переменной её значение кладётся на стек одним объектом `BulkPassive`:

Листинг 20 — Укладка переменной через `BulkPassive` ($O(1)$)

```

1 case refal.ast.Var v -> {
2     Expr<Term> val = bindings.get(v.key());
3     if (!val.isEmpty()) {
4         right.addFirst(new ViewTerm.BulkPassive(val)); //
5         O(1)
6     }
7 }
```

Когда обработчик `AngClose` снимает этот объект с `left`, он подмешивает его в накапливаемые аргументы вызовом `Expr.concat` — тоже амортизированно $O(1)$ (листинг 17).

Семантику это не трогает: `BulkPassive` встаёт в поле зрения ровно туда, где раньше лежали бы те же `Passive` в том же порядке. Меняется не порядок вычисления, а только способ хранения — одна ссылка вместо N объектов.

3.3.4 Укладка правой части правила

Когда сопоставление выбрало правило, на правый стек нужно положить его правую часть — но не как готовое значение, а так, чтобы оставшиеся в ней вызовы функций снова попали в нормализацию. Этим занимается `pushResultToStack`: он идёт по термам правой части справа налево (порядок важен, потому что стек разворачивает последовательность) и для каждого терма вызывает `pushTermToStack`.

Листинг 21 — Укладка терма правой части на стек (ветви `Var` и `Br`)

```

1 switch (t) {
2     case Var v -> {
3         Expr<Term> val = bindings.get(v.key());
4         if (!val.isEmpty())
5             right.addFirst(new ViewTerm.BulkPassive(val));
6     }
```

```

7      case Br br -> {
8          right.addFirst(ViewTerm.ParenClose.INSTANCE);
9          List<Term> items = br.items().toList();
10         for (int j = items.size() - 1; j >= 0; j--)
11             pushTermToStack(items.get(j), bindings, right)
12     ;
13         right.addFirst(ViewTerm.ParenOpen.INSTANCE);
14     }
15     default -> right.addFirst(new ViewTerm.Passive(t));
16 }

```

Каждый вид терма раскрывается по-своему. Переменная заменяется своим связанным значением: непустое значение кладётся на стек одним объектом `BulkPassive`. Скобочный терм (...) оборачивается маркерами `ParenOpen/ParenClose`, между которыми его содержимое рекурсивно укладывается тем же методом справа налево. Ветвь для вызова `<f ... >` в листинге опущена — она устроена симметрично `Br`: вместо скобочных маркеров ставятся `AngOpen(f)` и `AngClose`, между ними рекурсивно укладываются аргументы. Именно поэтому вложенные вызовы из правой части снова доходят до своего `AngClose` и нормализуются. Атом попадает в ветвь `default` и кладётся одним `Passive`. Так одна и та же укладка обслуживает и пассивные данные, и новые активные вызовы, не разделяя их на два разных пути.

Отдельного пояснения требует порядок обхода. И `pushResultToStack`, и обе рекурсивные ветви идут по термам справа налево (циклы со счётчиком от `size()-1` к нулю), а каждый терм кладётся на голову правого стека через `addFirst`. Стек по своей природе разворачивает последовательность, поэтому двойной разворот — обход с конца плюс вставка в начало — в итоге сохраняет исходный порядок термов в поле зрения. Если бы укладка шла слева направо, правая часть правила попала бы на стек в обратном порядке, и результат вычисления оказался бы зеркальным. Это тонкое место: для линейного представления порядок вставки очевиден, а на двустековой модели его приходится держать в голове, иначе ошибка проявится не падением, а молча искажённым результатом — что заметно труднее отлаживать.

Чтобы зацикленная или слишком долгая программа не висла бесконечно, каждый вход в `AngClose` проходит через счётчик шагов:

Листинг 22 — Контроль числа шагов нормализации

```
1 private void bumpStep() {  
2     steps++;  
3     if (steps > stepLimit)  
4         throw new RefalException("Step limit exceeded (" +  
5             stepLimit + ")");  
6 }
```

Один шаг — это одна редукция вызова функции. Входной вызов Go в счёт не идёт (счётчик стартует с -1), поэтому $\langle \text{Step} \rangle$ в программе и число шагов в сообщении об ошибке совпадают с интуитивным «сколько раз сработало правило». Порог `stepLimit` задаётся опцией `-step-limit`; по умолчанию — 100 000, чего хватает тестовым программам и заведомо мало для случайного закливания.

4 Аналитическая часть

В аналитической части собраны стратегия и состав автоматических тестов, метрики покрытия кода и числа сравнительных прогонов с двумя альтернативными реализациями языка. По ним проверяется две вещи: что реализация ведёт себя так, как задумано в конструкторской части (раздел 2), и что измеренная производительность совпадает с теоретическими ожиданиями из раздела 1.

4.1 Стратегия тестирования

Тесты разбиты на три уровня по характеру проверяемого поведения.

Юнит-тесты проверяют отдельные компоненты в изоляции: раскрученную двустороннюю очередь (`ShallowDeque`, `DeepDeque`, `ExprOps.concat`), лексер, парсер, семантический анализ, сопоставитель шаблонов, конструктор результата и встроенные функции.

Интеграционные тесты прогоняют полную цепочку «разбор → семантика → нормализация» на подготовленных фикстурах: фиксируется конкретный вывод интерпретатора и коды возврата CLI.

Тесты совместимости (`AutotestsRunnerTest`) запускают интерпретатор как внешний процесс на наборах тестовых программ, унаследованных от существующих реализаций Рефала, и проверяют результаты построчно.

В качестве фреймворка используется JUnit 5 [3]; все сравнения — через AssertJ [11]; покрытие кода — JaCoCo [13].

4.2 Состав тестов

Набор тестов собирался так, чтобы три уровня из п. 4.1 не дублировали, а дополняли друг друга. Юнит-тесты ловят ошибку как можно ближе к её источнику — в одном классе, на маленьком входе; интеграционные проверяют, что модули правильно стыкуются и вся цепочка от разбора до вывода работает как целое; тесты совместимости подтверждают, что интерпретатор ведёт себя как настоящая реализация Рефала на чужих программах. Отбирали тесты по двум принципам: покрыть все ветви, которые несут смысл (каждый класс переменных, каждый случай конкатенации, каждый вид ошибки), и закрепить регрессией те места, где уже находились дефекты.

Распределение тестов по классам приведено в таблице 5. Числа по классам `ShallowDequeTest–MainTest` получены прогоном `mvn test`; строка

AutotestsRunnerTest — количество внешних .ref-программ, на которых запускался интерпретатор.

Таблица 5 — Распределение тестов по классам

Тестовый класс	Уровень	Кол-во
refal.deque.ShallowDequeTest	юнит	11
refal.deque.DeepDequeTest	юнит	9
refal.deque.ConcatTest	юнит	9
refal.lexer.LexerTest	юнит	12
refal.parser.ParserTest	юнит	30
refal.semantic.SemanticTest	юнит	12
refal.runtime.MatcherTest	юнит	14
refal.runtime.SubstitutorTest	юнит	7
refal.runtime.BuiltinsTest	юнит	10
refal.ast.AstTest	юнит	7
refal.runtime.RuntimeIntegrationTest	интеграционный	12
refal.MainTest	интеграционный	10
refal.AutotestsRunnerTest	совместимость	81
Итого		224

4.2.1 Юнит-тесты раскрученной очереди

ShallowDequeTest проверяет: пустой буфер; заполнение до предела через pushFrontTight/pushBackTight; операции разделения separateFront/separateBack; защиту от переполнения (IllegalStateException при вызове tight-метода на полном буфере); переход в DeepDeque через публичный pushBack; корректность equals/hashCode для буферов с одинаковым содержимым, но разной позицией head в массиве.

DeepDequeTest проверяет построение глубокой очереди через расщепление мелкой; многократное переполнение, вызывающее рекурсивный рост поля mid; перенормировку mkDeep при уменьшении суммарного размера ниже CAPACITY; обход через iterator() в правильном порядке.

ConcatTest параметризован по четырём сценариям ExprOps.concat (SS, SD, DS, DD) и проверяет содержимое результата, его размер и тип (ShallowDeque при суммарном размере ≤ 9 , иначе DeepDeque).

4.2.2 Юнит-тесты лексера, парсера и семантики

LexerTest покрывает все классы токенов: ключевые слова, операторы, идентификаторы, переменные, строковые литералы в одинарных и двойных кавычках, целые числа со знаком, комментарии; учёт координат при многострочном вводе; исключения LexException при некорректных входах.

ParserTest содержит наибольшее число тестов (30): положительные случаи корректной грамматики; отрицательные случаи с ожидаемыми исключениями ParseException; параметризованные прогоны по фикстурам каталогов ok/ и bad-syntax/.

SemanticTest проверяет все виды семантических ошибок (SemanticException.Kind): повторное объявление функции, обращение к необъявленной функции, несоответствие типов переменной и др.

4.2.3 Юнит-тесты исполнения

MatcherTest (14 тестов) проверяет все классы переменных (s, t, e), повторные вхождения, перебор *e*-переменной с отказом и успехом, сопоставление со скобочными термами.

SubstitutorTest (7 тестов) проверяет правильное построение результата для всех видов узлов AST. Отдельно тестируется корректность при двукратном вхождении одной *e*-переменной в результат.

BuiltinsTest (10 тестов) покрывает каждую встроенную функцию: арифметику на макроцифрах, Compare, Lower/Upper, Symb/Numb, Mu.

4.2.4 Интеграционные тесты и тесты совместимости

Интеграционные тесты проверяют всю цепочку преобразований целиком, тогда как юнит-тесты изолируют отдельные модули. RuntimeIntegrationTest (12 тестов) прогоняет интерпретатор на отобранных фикстурах из src/test/resources/fixtures/ok/ и fixtures/desugared/: каждая фикстура — пара «программа.ref + ожидаемый вывод». Это сквозная проверка всей цепочки преобразований.

MainTest (10 тестов) проверяет CLI-обёртку: выбор точки входа, флаги — entry/—expr, корректные коды возврата (0/1/2/код от <Exit n>), захват вывода в stdout.

Тесты совместимости идут на шаг дальше интеграционных: они проверяют не отдельные фикстуры, а поведение интерпретатора на чужих наборах программ, изначально написанных для других реализаций Рефала. AutotestsRunnerTest запускает интерпретатор как внешний процесс на двух наборах тестовых программ.

Набор refal-5j (27 программ): 5 помечены как MUST_PASS; остальные используют синтаксис : (уточнение типа терма) или встроенные функции, не входя-

щие в реализованное подмножество (Random, GetEnv и др.), поэтому ожидаемо не проходят.

Набор `desugared` (50 программ): 18 помечены как `MUST_PASS`; ещё 11 программ проходят, но не обязательны (тестируют `built-in` вне нашего жёсткого ядра). Остальные требуют нереализованных встроенных (`SizeOf`, `GetEnv`, `Random`, `Time`, `Br/Dg` для файлового ввода/вывода), либо опираются на чтение существующих файлов.

Здесь важно различать два рода «непрохождения». Жёсткий отказ на программе из `MUST_PASS` означал бы ошибку реализации, и сборка такой отказ роняет. Мягкий отказ (`soft-fail`) на остальных программах — это не дефект, а граница подмножества: программа использует возможность, которая в базисном Рефале-5 сознательно не реализована. Таких возможностей две группы. Первая — расширенный синтаксис (уточнение типа терма через `:`, условия и блоки): по постановке он выносится во внешний рассаживатель, поэтому исходные, нерассаживаемые программы этого синтаксиса ядро и не обязано принимать. Вторая — встроенные функции вне сокращённой библиотеки (генерация случайных чисел, доступ к окружению и часам, часть файловых операций). Деление тестов на жёсткие и мягкие как раз и фиксирует эту границу в исполнимом виде: пополнение библиотеки или поддержка нового синтаксиса автоматически переводят соответствующие программы из мягких в проходящие, без правки самих тестов.

4.3 Покрытие кода

Отчёт JaCoCo генерируется командой `mvn verify` в каталог `target/site/jacoco/`. Покрытие по основным пакетам по инструкциям байт-кода и веткам приведено в таблице 6.

Таблица 6 — Покрытие кода по пакетам (JaCoCo)

Пакет	Инструкции	Ветки
refal.deque	86,7 %	72,9 %
refal.lexer	90,3 %	80,9 %
refal.parser	86,5 %	76,1 %
refal.semantic	94,8 %	95,5 %
refal.runtime	85,6 %	74,7 %
Итого по проекту	87,3 %	76,9 %

В `rom.xml` закреплены минимальные пороги покрытия: $\geq 70\%$ по инструкциям и $\geq 55\%$ по веткам для пакетов `refal.deque` и `refal.runtime`. Сборка `mvn`

verify падает, если порог нарушен; это страхует основные части реализации от регрессий.

Непокрытыми остаются защитные `IllegalStateException` в инвариантах структуры данных — они не достижимы при корректных входах, — а также ветви `equals/hashCode`, которые срабатывают только на маловероятных комбинациях позиций `head`.

4.4 Сравнительные прогоны

4.4.1 Условия измерений

Замеры проводились на одной рабочей машине: для честного сравнения важно не абсолютное быстродействие, а то, что все три реализации работают в одинаковых условиях — на том же процессоре, той же ОС и с тем же фоном. Поэтому конфигурация зафиксирована и приведена полностью (таблица 7), чтобы результаты можно было воспроизвести. Время намеренно измеряется как полное время жизни внешнего процесса, а не только «чистое» время счёта: именно столько ждёт пользователь, запустив программу из командной строки. У реализаций на JVM (наша и Refal-5J) в это время входит запуск виртуальной машины и прогрев JIT-компилятора, а у нативного Refal-5 PZ — только старт бинарника; такой режим ставит интерпретатор в заведомо невыгодные условия на малых входах, но зато отражает реальную картину использования.

Все прогоны выполнены в условиях, приведённых в таблице 7. Время фиксируется как полное время жизни внешнего процесса: запуск JVM, загрузка классов и собственно вычисление.

Таблица 7 — Условия сравнительных прогонов

Параметр	Значение
ЦПУ	Intel Core i5-11400H, 6 ядер / 12 потоков, 2,70 ГГц
ОЗУ	8 ГБ DDR4-3200
ОС	Windows 10 Pro 22H2 (сборка 19045)
JDK	Oracle JDK 25.0.3 LTS (HotSpot 25.0.3+9)
Refal-5 PZ	Version-PZ, релиз от 09.09.2025
Refal-5J	сборка от 13.03.2026

4.4.2 Тестовая программа и методика измерений

Тестовая программа (исходный код приведён в листинге 36 приложения Г) выполняет N итераций; каждая строит пару скобочных термов — (e.Асс A) или (e.Асс B) — из растущего аккумулятора e.Асс. Функция `Select` в зависимости от

текущего направления (Left/Right) возвращает одну из двух альтернатив; оставшийся терм следующего уровня присоединяется к аккумулятору. На каждом шаге аккумулятор *е.Асс* встречается в обоих аргументах, а значит, при построении результата его значение нужно присоединить к новому выражению.

Именно на этой конкатенации структуры данных расходятся. При связном списке *е.Асс* на *k*-й итерации имеет длину *k*, и каждый новый скобочный терм требует копирования всей цепочки: суммарно $1 + 2 + \dots + N = O(N^2)$. Раскрученная двусторонняя очередь выполняет ту же конкатенацию амортизированно за $O(1)$: значение переменной кладётся в стек нормализатора одной операцией *BulkPassive*, а сборка аргументов уже использует *Expr.concat* по схеме Окасаки [9]. Итоговая сложность цикла — $O(N)$.

Методика замеров для всех трёх реализаций одинакова. Для каждой реализации выполняется один прогревочный запуск (нужен для прогрева JIT-компилятора в Java-реализациях) и три измеряемых; в отчёт идёт медиана трёх измерений. Тайм-аут на один запуск — 120 секунд.

Размеры входа *N* покрывают два с половиной порядка: 100, 300, 600, 1000, 2000, 3000, 5000, 7000, 10 000, 15 000, 20 000.

Запуск выполняется скриптом `bench/loop-select-bench.ps1`:

Листинг 23 — Запуск бенчмарка Loop/Select

```
1 powershell -File bench\loop-select-bench.ps1 -WithPZ -  
   WithR5J
```

4.4.3 Результаты Loop/Select

Медианные времена выполнения приведены в таблице 8. Для каждого значения размера входа *N* приведено медианное время по трём прогонам в миллисекундах; меньшее значение соответствует более быстрому выполнению. Замеры охватывают диапазон *N* от 100 до 20 000, что позволяет проследить характер роста времени у каждой из трёх сравниваемых реализаций.

Те же данные представлены графически на рисунке 4: у нашей реализации зависимость времени от *N* ложится на прямую, у Refal-5 PZ и Refal-5J она растёт как парабола.

Таблица 8 — Время выполнения Loop/Select (мс, медиана 3 прогонов)

N	refal5q-impl	Refal-5 PZ	Refal-5J
100	122	13	96
300	132	13	88
600	144	15	95
1 000	156	17	103
2 000	186	27	107
3 000	211	41	133
5 000	224	80	186
7 000	236	142	274
10 000	254	277	440
15 000	297	632	830
20 000	302	1059	1348

Бенчмарк Loop/Select: сравнение асимптотики

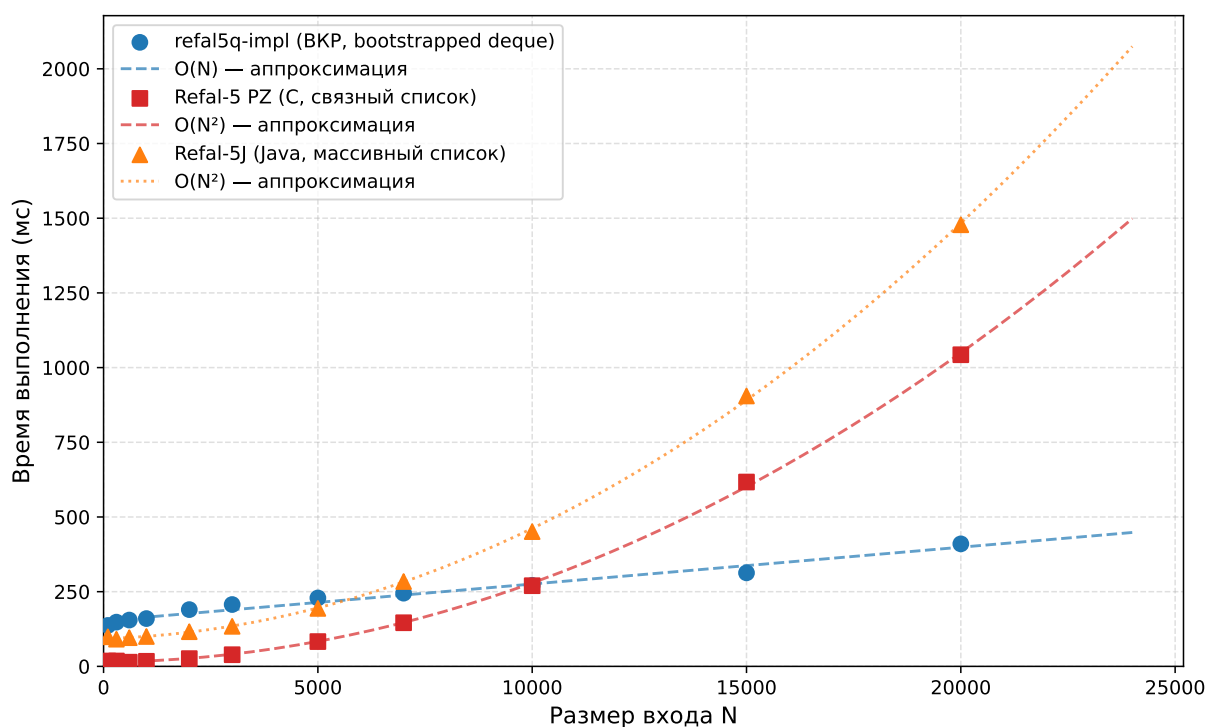
Асимптотика: наша реализация $O(N)$, связные списки $O(N^2)$

Рисунок 4 — Зависимость времени выполнения Loop/Select от размера входа N

4.4.4 Тест на реальной программе: рассахариватель

Loop/Select — синтетический тест: он нарочно нагружает ту единственную операцию, на которой расходятся структуры данных. Чтобы проверить реализацию на содержательной нагрузке, второй прогон выполнен на настоящей программе — рассахаривателе проекта refal-5-framework [4]. Это пять модулей (desugar, LibraryEx, R5FW-Parser, R5FW-Transformer, R5FW-Plainer) общим объёмом около

4200 строк; на вход ему подаётся R5FW-Parser.ref — сам синтаксический анализатор рассахаривателя, 1038 строк, дающий на выходе около 45 КБ текста.

Один и тот же рассахариватель прогоняется тремя способами:

- `refal5q-impl` — наш интерпретатор исполняет пять рассахаренных `.ref`-файлов напрямую;
- `Refal-5 PZ` — сначала компилирует `.ref` в `.rsl` (утилита `refc`), затем исполняет (утилита `refgo`);
- `Refal-5J` — компилирует `.ref` в `.java`, затем в `.class` (`r5jc + javac`) и запускает.

Все три реализации дают эквивалентный выход. Это и есть главный результат теста: наш интерпретатор корректно исполняет реальную программу такого объёма и воспроизводит поведение зрелых реализаций Рефала-5.

4.4.5 Методика и результаты рассахаривателя

Методика измерений совпадает с тестом `Loop/Select`: один прогревочный запуск и три измеряемых, в отчёт идёт медиана. Для `Refal-5 PZ` и `Refal-5J` компиляция выполняется один раз до замеров, поэтому измеряется только время исполнения скомпилированной программы. Тайм-аут на один запуск — 300 секунд.

Запуск выполняется скриптом `bench/desugar-bench.ps1`:

Листинг 24 — Запуск бенчмарка рассахаривателя

```
1 powershell -File bench\desugar-bench.ps1 -WithPZ -WithR5J
```

Медианные времена приведены в таблице 9.

Таблица 9 — Время рассахаривания `R5FW-Parser.ref` (мс, медиана 3 прогонов)

Реализация	Время, мс	Режим исполнения
<code>refal5q-impl</code>	2055	интерпретация
<code>Refal-5 PZ</code>	369	компиляция (<code>refc + refgo</code>)
<code>Refal-5J</code>	1632	компиляция (<code>r5jc + javac</code>)

4.4.6 Обсуждение результатов

Два прогона отвечают на разные вопросы, поэтому обсудим их по очереди: `Loop/Select` показывает, как ведёт себя структура данных под алгоритмической нагрузкой, а рассахариватель — как реализация справляется с реальной программой целиком.

Сначала о тесте `Loop/Select`. На малых N наша реализация проигрывает обоим конкурентам. Причина простая: запуск JVM и прогрев JIT-компилятора

стабильно стоят около 120 мс, и эта плата не зависит от размера входа. Refal-5 PZ — нативный бинарник, его старт укладывается в единицы миллисекунд; Refal-5J работает на той же JVM, что и мы, но тратит на инициализацию порядка 85–90 мс.

При $N \approx 7000$ кривая Refal-5J уже догоняет нашу, при $N \approx 9000$ её перешагивает Refal-5 PZ. После этой точки квадратичный рост конкурентов берёт своё: на $N = 10\,000$ PZ уже медленнее (277 мс против 254 мс), а на $N = 20\,000$ разрыв с PZ становится 3,5-кратным (1059 мс против 302 мс) и 4,5-кратным с Refal-5J (1348 мс против 302 мс).

Объяснение различия лежит в способе хранения объектного выражения. Refal-5 PZ и Refal-5J держат выражение в виде двусвязного списка термов: при построении нового скобочного терма (е.Асс A/B) на k -й итерации нужно скопировать всю накопленную часть длиной k . N итераций по $O(k)$ дают $O(N^2)$. Наш интерпретатор устроен иначе: значение переменной идёт на стек одним объектом BulkPassive, а сборка аргументов вызова использует Expr.concat — по схеме Окасаки это амортизированная константа независимо от длины операндов. N итераций по $O(1)$ дают $O(N)$ — ровно то, что показывает прямая на графике.

У замера есть границы применимости, и их стоит назвать прямо. Меряется время жизни всего процесса, поэтому наши ≈ 120 мс на $N = 100$ — это почти целиком запуск JVM, а не работа интерпретатора. Диапазон N до 20 000 захватывает лишь начало параболы конкурентов: на больших N разрыв рос бы быстрее, но прогон при $N \sim 10^5$ уже упирается в тайм-аут 120 с. Наконец, PZ и Refal-5J в обоих тестах выступают как компиляторы — компиляция сделана заранее, и замеряется только исполнение готового кода, так что на Loop/Select мы сравниваем алгоритмические сложности структур данных, а не режимы исполнения.

Теперь о тесте рассаживателя. И здесь конкуренты также работают как компиляторы: и Refal-5 PZ, и Refal-5J сначала переводят программу в нативный код или байт-код, а потом исполняют уже скомпилированное. Отличие от Loop/Select в том, что рассаживатель не содержит квадратичного паттерна, поэтому алгоритмического преимущества у нас нет, и они закономерно быстрее нашего интерпретатора — Refal-5 PZ в 5,6 раза, Refal-5J в 1,3 раза. Это не алгоритмический проигрыш, а ожидаемая разница между компиляцией и интерпретацией: наша реализация на каждом шаге заново разбирает поле зрения, тогда как

скомпилированная программа исполняет готовый код. Наибольший отрыв даёт Refal-5 PZ как нативный компилятор; Refal-5J ближе к нам, потому что работает на той же JVM и платит те же стартовые накладные расходы.

Важно, что главный вывод этого теста не про время, а про корректность: наш интерпретатор исполняет программу в 4200 строк и выдаёт ровно тот же результат, что зрелые реализации Рефала-5. Вместе два прогона дают согласованную картину. На алгоритмически тяжёлом паттерне, где растущий аккумулятор многократно копируется, выигрывает структура данных — раскрученная очередь даёт $O(N)$ там, где списочные представления дают $O(N^2)$. На обычной программе без такого паттерна решает способ исполнения, и компиляция ожидаемо обходит интерпретацию. Ни одно из наблюдений не противоречит другому: они измеряют разные свойства реализации.

ЗАКЛЮЧЕНИЕ

В рамках выпускной квалификационной работы поставлена и решена задача разработки компилятора базисного подмножества Рефала-5 с представлением объектных выражений на основе раскрученной двусторонней очереди, а также подтверждения корректности и работоспособности реализации автоматическими тестами и сравнительным прогоном с двумя альтернативными реализациями языка.

В исследовательской части рассмотрены базовые понятия Рефала-5 и существующие представления объектных выражений (классическое плоское двусвязное, компромиссное с подвешенными скобками, массивное, векторно-списковое и дерево конкатенаций), обсуждены их компромиссы по ключевым операциям и обоснован выбор раскрученной двусторонней очереди по Окасаки [9] как структуры, амортизированно покрывающей все основные операции (отделение термина с концов, конкатенацию и многократное использование значения переменной).

В конструкторской части зафиксированы функциональные и нефункциональные требования к системе, декомпозиция на модули с привязкой к Java-пакетам, интерфейс контейнера выражения и требования к механизму сопоставления, выделены точки расширения и определены классы ошибок с единым форматом диагностических сообщений.

В технологической части реализован компилятор с интерпретирующим исполнением, выполняющий лексический, синтаксический и семантический анализ исходного текста и формирующий внутреннее представление программы. Раскрученная двусторонняя очередь реализована через запечатанный интерфейс $\text{Expr}\langle T \rangle$ и пару классов $\text{ShallowDeque}\langle T \rangle$ (циклический массив фиксированной ёмкости $\text{CAPACITY} = 9$) и $\text{DeepDeque}\langle T \rangle$ (тройка $\langle \text{front}, \text{mid}, \text{back} \rangle$ с рекурсивным средним уровнем); конкатенация со всеми четырьмя случаями $\text{Shallow/Deep} \times \text{Shallow/Deep}$ вынесена в служебный класс ExprOps . Исполняющий модуль реализован как цикл нормализации в прикладном порядке с сопоставлением образца через перебор длины e -переменной и сборкой результата через Expr.concat . Реализован минимальный набор встроенных функций (ввод-вывод, арифметика, работа со строками и символами, функции высшего порядка), достаточный для тестовых программ и для выхода рессахаривателя `r5fw-desugar`

проекта `refal-5-framework` [4]; директива `$EXTERN` поддерживается, что обеспечивает совместимость с этим рессахаривателем.

В аналитической части построен автоматический набор из 224 тестов: 143 теста JUnit 5 [3], покрывающих лексер, парсер, семантический анализ, сопоставление, подстановку, цикл нормализации и саму раскрученную очередь, и 81 внешняя программа в рамках `AutotestsRunnerTest` — фикстуры из репозитория `refal-5j` и `refal-5-framework` [4]. Среднее покрытие по инструкциям байт-кода составляет около 87 %, по веткам — около 77 %, что выше зафиксированных в `rom.xml` порогов. Дополнительно проведены два сравнительных прогона трёх реализаций Рефала-5 (`refal5q-impl`, `Refal-5 PZ` [1] и `Refal-5J`): синтетический тест `Loop/Select` и прогон рессахаривателя `R5FW-Parser.ref` из проекта `refal-5-framework` [4]. Замеры автоматизированы скриптами `bench/loop-select-bench.ps1` и `bench/desugar-bench.ps1`; методика — один прогревочный запуск и три измеряемых, в отчёт идёт медиана.

Основные численные результаты: автоматический прогон `mvn clean verify` проходит без нарушений; контроль стиля `Checkstyle` и покрытие выполняются выше установленных порогов. На тесте `Loop/Select` на малых N разработанная реализация проигрывает конкурентам из-за накладных расходов запуска JVM (≈ 120 мс), однако при росте входа зависимость времени от N остаётся линейной: при $N = 20\,000$ время составляет 302 мс против 1059 мс у `Refal-5 PZ` и 1348 мс у `Refal-5J` (разрыв 3,5 и 4,5 раза соответственно). На прогоне рессахаривателя все три реализации выдают эквивалентный результат; по времени `Refal-5 PZ` быстрее в 5,6 раза, `Refal-5J` — в 1,3 раза, что соответствует ожидаемому преимуществу компиляции над интерпретацией на программе без квадратичного паттерна.

Возможные направления дальнейшего развития системы:

- Компиляция в JVM-байт-код по образцу `Refal-5J`: порождение для каждого правила отдельного Java-метода уменьшило бы постоянный множитель и сократило бы отставание от `Refal-5J` на программах без квадратичного паттерна, например на прогоне рессахаривателя.
- Поддержка модульной системы Рефала-5 (директива `*$FROM`, квалифицированные имена). Текущая реализация поддерживает конкатенацию нескольких файлов через синтаксис `+`, однако `*$FROM` и квалифицированные имена функций не реализованы.

- Расширение библиотеки встроенных функций до полного набора Рефал-5 (Br/Dg, шифр-функции, файловый ввод-вывод). Архитектура регистрации позволяет это сделать без изменений ядра.

Полученная программная система (Maven-проект `refal5q-impl` объёмом $\sim 3\,800$ строк основного кода и $\sim 1\,700$ строк тестов) может использоваться как самостоятельный компилятор базисного подмножества Рефала-5 и как базовая платформа для дальнейших исследований в области эффективных представлений объектных выражений и компиляции функциональных языков переписывания термов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. REFAL-5 home page [Электронный ресурс]. — URL: <http://www.botik.ru/pub/local/scp/refal5/> (дата обр. 20.04.2026) ; Официальный сайт дистрибутива Refal-5 PZ (реализация на С, байт-код RASL).
2. Apache Maven Project: Documentation [Электронный ресурс]. — URL: <https://maven.apache.org/guides/index.html> (дата обр. 05.05.2026).
3. JUnit 5 User Guide [Электронный ресурс]. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обр. 05.05.2026).
4. Refal-5-framework: набор библиотек для преобразования программ на Рефале-5 [Электронный ресурс]. — URL: <https://github.com/Mazdaywik/refal-5-framework> (дата обр. 15.04.2026).
5. REFAL-5: Programming Guide & Reference Manual [Электронный ресурс]. — URL: <http://refal.botik.ru/book/html/> (дата обр. 28.03.2026).
6. Документация Refal-05 [Электронный ресурс]. — URL: <https://mazdaywik.github.io/Refal-05/> (дата обр. 03.04.2026).
7. Скоробогатов С. Ю. Реализация Рефал-компилятора. Представление данных и язык сборки. — 2008. — URL: <https://bmstu-iu9.github.io/> (дата обр. 05.05.2026) ; Рукопись.
8. Абрамов С. М., Романенко С. А. Представление объектных выражений массивами при реализации языка Рефал : тех. отч. / ИПМ им. М. В. Келдыша АН СССР. — 1988. — № 186.
9. Okasaki C. Purely Functional Data Structures. — Cambridge University Press, 1998.
10. The Java Language Specification, Java SE 25 Edition [Электронный ресурс] / J. Gosling [и др.]. — URL: <https://docs.oracle.com/javase/specs/jls/se25/html/index.html> (дата обр. 05.05.2026).
11. AssertJ – fluent assertions for Java [Электронный ресурс]. — URL: <https://assertj.github.io/doc/> (дата обр. 05.05.2026).
12. Checkstyle: a development tool for checking Java source code [Электронный ресурс]. — URL: <https://checkstyle.sourceforge.io/> (дата обр. 05.05.2026).

13. JaCoCo Java Code Coverage Library [Электронный ресурс]. — URL: <https://www.jacoco.org/jacoco/trunk/doc/> (дата обр. 05.05.2026).

ПРИЛОЖЕНИЕ А

Формальная грамматика базисного подмножества Рефала-5

Грамматика приведена в нотации EBNF. Синтаксис базисного подмножества является точным подмножеством синтаксиса Рефала-5. В реализуемом базисном подмножестве дополнительно поддерживается директива \$EXTERN, нужная для согласованности с выходом рессахаривателя r5fw-desugar [4].

Лексика

- Пробелы и переводы строк разделяют токены и игнорируются.
- Комментарий: строка, начинающаяся с *, до конца строки.
- Целое число: -?[0-9]+.
- Строка: '...' либо "... " (без экранирования; допускаются любые символы до следующей кавычки того же вида).
- Идентификатор: [A-Za-z._\$][A-Za-z0-9._\$]*.
- Зарезервированные директивы: \$ENTRY, \$EXTERN.
- Скобки и разделители: { } () < > = ; , .

Синтаксис (EBNF)

Листинг 25 — EBNF-грамматика базисного подмножества Refal-5

```
1 Program      ::= TopLevel* EOF ;
2
3 TopLevel     ::= EntryDecl | ExternDecl | FuncDef ;
4
5 EntryDecl    ::= "$ENTRY" Ident ";"
6               | "$ENTRY" FuncDef ;
7
8 ExternDecl   ::= "$EXTERN" Ident ("," Ident)* ";" ;
9
10 FuncDef     ::= Ident "{" Rule+ "}" ;
11
12 Rule        ::= Pattern "=" Result ";"? ;
13
14 Pattern     ::= PatternTerm* ;
15 PatternTerm ::= Int | Str | QuotedSym | Var | "("
16               PatternTerm* ")" ;
17
17 Result      ::= ResultTerm* ;
18 ResultTerm  ::= Int | Str | QuotedSym | Var | Symbol
```

```

19         | "(" ResultTerm* ")"
20         | "<" Ident ResultTerm* ">" ;
21
22 Var      ::= ("s" | "t" | "e") "." VarName ;
23 VarName  ::= (Letter | Digit | "_" | "." | "$")+ ;
24
25 Symbol   ::= Ident ;
26 Ident    ::= Letter (Letter | Digit | "_" | "-" | "+" |
        ".")* ;
27 Int      ::= Digit+ ;
28 Str      ::= "'" Char* "'" ;
29 QuotedSym ::= '"' Char* '"' ;

```

Семантика сопоставления

- $s.X$ — связывается с ровно одним атомарным термом (символ, число или строка); скобочный терм не подходит.
- $t.X$ — связывается с ровно одним термом (включая скобочный).
- $e.X$ — связывается с последовательностью термов (возможно пустой).
- Если переменная встречается в правиле несколько раз — все вхождения должны быть связаны с одинаковыми значениями.
- В теле функции выбирается первое правило, чей образец сопоставим с аргументами; затем результат подставляется и редуцируется.

Что не поддерживается

- Условия , $e.Cond$ после паттерна, блоки $\{ : \}$ — для них применяется внешний рессахариватель `r5fw-desugar`, выход которого уже укладывается в эту грамматику.
- Экранирование в строках (`\n`, `\x42`).
- Макроцифры `@BIG-MACRODIGIT@`.
- Псевдокомментарии `*$CLASSIC / *$EXTENDED`.
- Декларации `*$FROM` (директива модульной системы Рефала-5); несколько файлов загружаются через синтаксис `+: java -jar target/refal5q.jar a.ref+b.ref+c.ref`.

ПРИЛОЖЕНИЕ Б

Листинги ключевых функций

В приложении приведены ключевые фрагменты исходного кода Java-реализации с пояснительными комментариями. Каждый листинг сопровождается ссылкой на соответствующий файл в каталоге `refal5q-impl/src/main/java/refal/`.

Конкатенация контейнеров `ExprOps.concat`

Файл `refal5q-impl/src/main/java/refal/deque/ExprOps.java`.

Листинг 26 — Конкатенация двух `Expr` с разбором всех четырёх случаев

```
1 static <T> Expr<T> concat(Expr<T> a, Expr<T> b) {
2     if (a.isEmpty()) return b;
3     if (b.isEmpty()) return a;
4     if (a instanceof ShallowDeque<T> sa && b instanceof
5         ShallowDeque<T> sb) return concatSS(sa, sb);
6     if (a instanceof ShallowDeque<T> sa && b instanceof
7         DeepDeque<T> db) return concatSD(sa, db);
8     if (a instanceof DeepDeque<T> da && b instanceof
9         ShallowDeque<T> sb) return concatDS(da, sb);
10    DeepDeque<T> da = (DeepDeque<T>) a;
11    DeepDeque<T> db = (DeepDeque<T>) b;
12    return concatDD(da, db);
13 }
14
15 private static <T> Expr<T> concatDD(DeepDeque<T> a,
16     DeepDeque<T> b) {
17     int total = a.size() + b.size();
18     ShallowDeque<T> ab = a.back();
19     ShallowDeque<T> bf = b.front();
20     int boundary = ab.size() + bf.size();
21
22     Expr<ShallowDeque<T>> aMidPlus = a.mid();
23     if (boundary == 0) {
24         // оба граничных буфера пусты - ничего добавлять не нужно.
25     } else if (boundary <= CAPACITY) {
26         ShallowDeque<T> merged = ShallowDeque.empty();
27         for (T x : iter(ab)) merged = merged.pushBackTight
28             (x);
29         for (T x : iter(bf)) merged = merged.pushBackTight
30             (x);
```

```

25         aMidPlus = aMidPlus.pushBack(merged);
26     } else {
27         Object[] tmp = new Object[boundary];
28         int i = 0;
29         for (T x : iter(ab)) tmp[i++] = x;
30         for (T x : iter(bf)) tmp[i++] = x;
31         int rightLen = boundary - CAPACITY;
32         if (rightLen < ShallowDeque.MIN_FILL) rightLen =
ShallowDeque.MIN_FILL;
33         int leftLen = boundary - rightLen;
34         ShallowDeque<T> m1 = ShallowDeque.ofArray(tmp, 0,
leftLen);
35         ShallowDeque<T> m2 = ShallowDeque.ofArray(tmp,
leftLen, rightLen);
36         aMidPlus = aMidPlus.pushBack(m1).pushBack(m2);
37     }
38
39     Expr<ShallowDeque<T>> newMid = concat(aMidPlus, b.mid
());
40     return DeepDeque.mkDeep(a.front(), newMid, b.back(),
total);
41 }

```

Случай Deep+Deep явно показан полностью — в нём проявляется рекурсивный вызов concat на средних очередях буферов, обеспечивающий амортизированной $O(1)$ сложность по Окасаки [9]. Случаи Shallow+Shallow, Shallow+Deep и Deep+Shallow (методы concatSS, concatSD, concatDS) реализованы аналогично с расщеплением объединённого буфера ровно пополам, если он превышает CAPACITY.

Сопоставление образца с выражением `Matcher.match`

Файл `refal5q-impl/src/main/java/refal/runtime/Matcher.java`.

Листинг 27 — Сопоставление образца с выражением (с backtracking по е-переменным)

```

1 public static Optional<Bindings> match(List<Term> pattern,
    List<Term> expr) {
2     return rec(pattern, 0, expr, 0, expr.size(), Bindings.
empty());
3 }

```

```

4
5 private static Optional<Bindings> rec(
6     List<Term> pat, int pi,
7     List<Term> exp, int ei, int eEnd,
8     Bindings env) {
9     if (pi == pat.size()) {
10         return ei == eEnd ? Optional.of(env) : Optional.
empty();
11     }
12     Term p = pat.get(pi);
13
14     if (p instanceof Var v && v.kind() == 'e') {
15         String key = v.key();
16         if (env.contains(key)) {
17             // Повторное вхождение: проверяем совпадение.
18             List<Term> bound = env.get(key);
19             int len = bound.size();
20             if (ei + len > eEnd) return Optional.empty();
21             for (int k = 0; k < len; k++) {
22                 if (!exp.get(ei + k).equals(bound.get(k)))
return Optional.empty();
23             }
24             return rec(pat, pi + 1, exp, ei + len, eEnd,
env);
25         }
26         // Свежая e.X - перебираем длину захвата 0, 1, ...
27         for (int len = 0; ei + len <= eEnd; len++) {
28             List<Term> chunk = exp.subList(ei, ei + len);
29             Bindings env1 = env.with(key, List.copyOf(
chunk));
30             Optional<Bindings> r = rec(pat, pi + 1, exp,
ei + len, eEnd, env1);
31             if (r.isPresent()) return r;
32         }
33         return Optional.empty();
34     }
35
36     if (ei >= eEnd) return Optional.empty();
37     Term x = exp.get(ei);
38

```

```

39     if (p instanceof Var v) {
40         String key = v.key();
41         if (v.kind() == 's') {
42             if (x instanceof Br) return Optional.empty();
43             if (env.contains(key)) {
44                 List<Term> bound = env.get(key);
45                 if (bound.size() != 1 || !bound.get(0).
equals(x)) return Optional.empty();
46                 return rec(pat, pi + 1, exp, ei + 1, eEnd,
env);
47             }
48             return rec(pat, pi + 1, exp, ei + 1, eEnd, env
.with(key, List.of(x)));
49         }
50         if (v.kind() == 't') {
51             if (env.contains(key)) {
52                 List<Term> bound = env.get(key);
53                 if (bound.size() != 1 || !bound.get(0).
equals(x)) return Optional.empty();
54                 return rec(pat, pi + 1, exp, ei + 1, eEnd,
env);
55             }
56             return rec(pat, pi + 1, exp, ei + 1, eEnd, env
.with(key, List.of(x)));
57         }
58     }
59
60     if (p instanceof Br pBr) {
61         if (!(x instanceof Br xBr)) return Optional.empty
();
62         Optional<Bindings> inner = rec(pBr.items(), 0, xBr
.items(), 0, xBr.items().size(), env);
63         if (inner.isEmpty()) return Optional.empty();
64         return rec(pat, pi + 1, exp, ei + 1, eEnd, inner.
get());
65     }
66
67     if (p.equals(x)) return rec(pat, pi + 1, exp, ei + 1,
eEnd, env);
68     return Optional.empty();

```

Подстановка результата правила Substitutor.substitute

Файл refal5q-impl/src/main/java/refal/runtime/Substitutor.java.

Листинг 28 — Сборка результата правила через Expr.concat

```

1 public static Expr<Term> substitute(List<Term> result,
   Bindings env) {
2     Expr<Term> out = Expr.empty();
3     for (Term t : result) {
4         switch (t) {
5             case Var v -> {
6                 String key = v.key();
7                 if (!env.contains(key)) {
8                     throw new IllegalStateException(
9                         "Unbound variable in result: "
10                        + key);
11                 }
12                 out = Expr.concat(out, Expr.fromList(env.
13                     get(key)));
14             }
15             case Br br -> {
16                 Expr<Term> inner = substitute(br.items(),
17                     env);
18                 out = out.pushBack(new Br(inner.toList()))
19                 ;
20             }
21             case Call c -> {
22                 Expr<Term> argDq = substitute(c.args(),
23                     env);
24                 out = out.pushBack(new Call(c.fn(), argDq.
25                     toList()));
26             }
27             default -> out = out.pushBack(t);
28         }
29     }
30     return out;
31 }

```

Использование switch-выражения с шаблонами над запечатанным интерфейсом Term обеспечивает компилируемую полноту разбора: если в иерархию AST добавляется новый вид терма, компилятор Java потребует обработать его явно.

Цикл нормализации **Runtime.normalize** и **Runtime.evalCall**

Файл `refal5q-impl/src/main/java/refal/runtime/Runtime.java`.

Листинг 29 — Цикл нормализации (прикладной порядок, сначала крайний слева)

```
1 public Expr<Term> normalize(Expr<Term> expr) {
2     Expr<Term> out = Expr.empty();
3     for (Term t : expr) {
4         out = Expr.concat(out, normalizeTerm(t));
5     }
6     return out;
7 }
8
9 private Expr<Term> normalizeTerm(Term t) {
10     return switch (t) {
11         case Call c -> {
12             bumpStep();
13             Expr<Term> normalizedArgs = normalize(Expr.
14 fromList(c.args()));
15             Expr<Term> body = evalCall(c.fn(),
16 normalizedArgs.toList());
17             yield normalize(body);
18         }
19         case Br br -> {
20             Expr<Term> inner = normalize(Expr.fromList(br.
21 items()));
22             yield Expr.singleton(new Br(inner.toList()));
23         }
24         default -> Expr.singleton(t);
25     };
26 }
27
28 public Expr<Term> evalCall(String fn, List<Term> args) {
29     BuiltinFn b = builtins.get(fn);
30     if (b != null) {
31         List<Term> r = b.apply(this, args);
32         return Expr.fromList(r);
33     }
34 }
```

```
31     Optional<Func> f = program.getFunc(fn);
32     if (f.isEmpty()) {
33         throw new RefalException("Unknown function: " + fn
34     );
35     }
36     for (Rule rule : f.get().rules()) {
37         var bindings = Matcher.match(rule.pattern(), args)
38     ;
39         if (bindings.isPresent()) {
40             return Substitutor.substitute(rule.result(),
41             bindings.get());
42         }
43     }
44     throw new RefalException("No matching rule for " + fn
45     + " on args=" + args);
46 }
```

ПРИЛОЖЕНИЕ В

Команды воспроизведения тестов и сравнительного прогона

Все команды выполняются из каталога `refal5q-impl/`. Предполагаются установленными Java 25 LTS [10] (с переменной `JAVA_HOME`, указывающей на JDK) и Apache Maven 3.9 или выше [2].

Установка инструментов

Листинг 30 — Проверка инструментов перед сборкой

```
1 $ java -version          # ожидается 25.x
2 $ javac -version         # ожидается 25.x
3 $ mvn -version           # ожидается 3.9.x или новее
```

Зависимости проекта (JUnit 5, AssertJ, JaCoCo, Checkstyle) подтягиваются Maven автоматически из локального репозитория `~/.m2/`; явная установка не требуется.

Полная сборка с тестами и проверками

Листинг 31 — Полная сборка проекта

```
1 $ mvn clean verify
```

Команда выполняет:

- компиляцию (`maven-compiler-plugin`, целевой релиз — 25);
- запуск всех 143 тестов JUnit 5 (`maven-surefire-plugin`);
- построение самодостаточного JAR-файла `target/refal5q.jar` (`maven-shade-plugin`);
- проверку покрытия (`jacoco-maven-plugin`; пороги $\geq 70\%$ по инструкциям и $\geq 55\%$ по веткам для пакетов `refal.deque` и `refal.runtime`);
- проверку стиля кода (`maven-checkstyle-plugin`, конфигурация `checkstyle.xml`).

При успешном завершении в каталоге `target/site/jacoco/` лежит HTML-отчёт о покрытии, в `target/` — jar-артефакт.

Запуск программы

Листинг 32 — Запуск программы из файла

```
1 $ java -jar target/refal5q.jar bench/hello.ref --expr "'
    World'"
```

При наличии \$ENTRY в файле опция `-entry` необязательна. Опция `-step-limit N` устанавливает лимит шагов редукции (по умолчанию 100 000).

Workflow с внешним рассахаривателем

Если исходный файл написан с использованием расширенного синтаксиса Рефала-5 (условия, `e.Cond`, блоки `{ : }`), то перед запуском его пропускают через рассахариватель из проекта `refal-5-framework` [4]:

Листинг 33 — Полный workflow с рассахаривателем

```
1 $ refgo r5fw-desugar program.ref program-desugared.ref
2 $ java -jar target/refal5q.jar program-desugared.ref --
   entry Go
```

Сравнительные прогоны трёх реализаций Рефала-5

Сравнительные прогоны из подраздела 4.4 автоматизированы двумя PowerShell-скриптами. Оба ожидают установленный Refal-5 PZ [1] (дистрибутив Version PZ с сайта ИПИМ им. А. П. Ершова; пути к `refgo.exe` и `refc.exe` задаются в начале скрипта) и опционально — Refal-5J при включённой опции `-WithR5J`.

Скрипт `bench/loop-select-bench.ps1` прогоняет синтетический тест Loop/Select при значениях N от 100 до 20 000:

Листинг 34 — Запуск бенчмарка Loop/Select

```
1 $ powershell -File bench\loop-select-bench.ps1 -WithPZ -
   WithR5J
```

Скрипт `bench/desugar-bench.ps1` измеряет время рассахаривания `R5FW-Parser.ref`:

Листинг 35 — Запуск бенчмарка рассахаривателя

```
1 $ powershell -File bench\desugar-bench.ps1 -WithPZ -
   WithR5J
```

По умолчанию выполняется 1 прогревочный запуск и 3 измеряемых, в отчёт идёт медиана; параметры регулируются опциями `-WarmupRuns` и `-MeasuredRuns`. Результаты Loop/Select сохраняются в `bench/loop-select-results.csv`; график для основного текста строится отдельно командой `python bench/plot-benchmark.py -out bench/loop-select-graph.pdf`. Время рассахаривания записывается в `bench/desugar-results.csv`.

ПРИЛОЖЕНИЕ Г

Исходный код теста Loop/Select

Тестовая программа сравнительного прогона из подраздела 4.4. Файл `refal5q-impl/bench/loop-select.ref`. Аргумент `<Arg 1>` — число итераций N ; на каждой итерации функция `Loop` строит новый скобочный терм из растущего аккумулятора `e.Acc`, а `Select` выбирает одну из двух альтернатив в зависимости от направления.

Листинг 36 — Тестовая программа Loop/Select

```
1 $ENTRY Go {
2   = <TimeElapsed 0>
3   <Loop Left () <Numb <Arg 1>>>
4   <Prout <TimeElapsed>>;
5 }
6
7 Loop {
8   s.Side (e.Acc) 0 = /* done */;
9   s.Side (e.Acc) s.Rest
10  = <Loop <Select s.Side (e.Acc A) (e.Acc B)> <- s.Rest
    1>>;
11 }
12
13 Select {
14   Left  t.Left t.Right = Right t.Left;
15   Right t.Left t.Right = Left  t.Right;
16 }
```